University of Liverpool
Department of Computer Science
COMP702 MSc Computer Science Project

Kodro

An Offline, Self-Refining Platform for Designing Robots and Validating Their Behaviour in Realistic Simulation

---

Vaibhav Lalwani
Student ID: 201979723

Supervisor: Keith Dures

Submitted in partial fulfilment of the requirements for the degree of
Master of Science in Computer Science

11 September 2026

Word count excludes the title page, contents, references and appendices.

Declaration

I confirm that this dissertation is my own work and that it has not been submitted, in whole or in part, for any other degree or qualification. Where I have drawn on the work or ideas of others, the source is acknowledged in the text and listed in the references. The software described here was built by me over the project period and is held in a version controlled repository, so every technical claim in this document can be checked against the code at the tag it describes.

Use of generative artificial intelligence. Generative artificial intelligence assisted the development of the software and the drafting of this document, used in line with University guidance on acknowledging such assistance. I directed the work, made every design decision, and verified every result reported here. All technical claims describe the working system as built and tested. Every figure that reports a measurement is reported as measured, with the method and its limits stated plainly. No result from a human study is reported, because that study has not yet been run, and the text says so wherever it is relevant rather than implying evidence that does not exist. The teacher-persona walkthrough in the evaluation chapter is an analyst-written design speculation and is labelled as such, not as data. The references were checked at the time of writing.

Ethics. The work to date uses simulated personas only, with no human participants and no personal data (ethics data category A0, participant category 2). Any later study with real users will be agreed with my supervisor and run under appropriate informed consent before it begins.

Abstract

A person with an idea for a robot has no safe, cheap way to know how it would perform. Buying parts, wiring a board and writing control code is slow, and the first time a real robot meets the real world it tends to drive into something. Kodro is an offline desktop proving ground that closes that gap. A skeptical builder imports a real robot specification, its motor, battery, body and sensors as measured numbers, and watches how that machine would behave in a realistic simulated world, with every reported figure carried at a stated level of fidelity rather than dressed up as a certified result. The imported specification drives the simulation, so the numbers on screen follow from the hardware: a heavier build accelerates more slowly and drains its battery faster, a stronger motor raises the closed-form top speed, and a sensor that is not fitted withholds its command from every way of programming the robot. The same platform doubles as a design studio in which a non-expert assembles a robot from a parts catalogue and programs it in Python or in blocks, so the tool serves both the builder validating a real machine and the learner exploring an idea.

The honesty is structural, not editorial. Every performance figure is badged in one of three tiers, HONOURED where the simulation runs the measured number, APPROXIMATED where a first-order model stands in for it, and NOT SIMULATED where a value is reported but never driven, and a per-robot verification report lays the whole build out this way for a builder to keep. One shared motion model backs the simulation: a single set of closed-form equations and constants, mirrored in JavaScript and in Python and locked together by a constants hash, a golden-trace corpus and a formula-parity test, so the two engines cannot silently drift. A small language model, run only on the local machine through Ollama, grounds its suggestions in the build and refines its advice from accumulated use through a reflection loop and a reusable skill library, without retraining the model and without reaching the internet. When no model is present the platform falls back within a fixed time budget to a deterministic rule engine, so it stays fully usable with the network off, and a deterministic validator, not the model, always has the final word on whether a program is safe to run.

This dissertation states the problem and the aims, reviews the literature the design follows from, sets out the requirements and the layered architecture, describes the implementation against the real codebase, and reports an honest evaluation. The interpreter and its kinematics pass a functional quality harness at one hundred and eighty of one hundred and eighty checks, and the Python engine carries a suite of more than one thousand tests above the eighty-five percent coverage gate the suite enforces. A headless regression net exercises the interface, an objective, execution-scored persona-task evaluation puts a reproducible figure on the offline assistant, and a seeded grounding benchmark measures whether a local model's generated programs stay inside the build's real command set. A single-rater formative review across simulated personas traces an improving usability trend as defects were fixed. No result from a study with real users is reported, because that study has deliberately been left as future work, and the text says so wherever it is relevant. The contribution is not a new algorithm. It is a working demonstration that a real specification import, a disciplined three-tier fidelity disclosure, retrieval grounding, a sandboxed simulation gate, an experience memory and a skill library combine into an honest proving ground that runs entirely on one machine, together with a measured account of how far offline self-refinement can go when changing the model weights is off the table.

Acknowledgements

I thank my supervisor, Keith Dures, for steady guidance and for keeping the scope honest. I thank the maintainers of the open-source libraries this work stands on, named in the implementation chapter. The framing of the project owes a debt to a long line of work on learning by building, on grounding language models in what a system can actually do, and on closing the gap between simulation and reality, all of which is acknowledged in the references.

Contents

Chapter 1

Introduction

1.1 Context and motivation

A robot is one of the few pieces of software that can hurt someone if it is wrong. A spreadsheet that miscalculates wastes time. A control program that misjudges a distance drives a machine into a wall, or into a person. This is the reason the gap between an idea for a robot and a working robot is wide, and it is why the gap is expensive to cross. Parts cost money, wiring takes time and care, and the first contact between a freshly written control program and the physical world is usually a small disaster. People who are not professional roboticists, the makers, the hobbyists, the students, the curious, feel this barrier most sharply. They have the idea and the willingness, and they lack a safe and cheap place to try the idea before they commit money and hardware to it.

The obvious answer is simulation. Try the robot in software first, watch it behave, fix what is wrong, and only then build it. This is sound, and it is exactly what professional roboticists do with heavyweight simulators. The difficulty is that those simulators are heavyweight in every sense. They assume a workstation, a graphics card, an internet connection for assets and accounts, and an operator who already knows robotics. They are built for the expert who is refining a known design, not for the non-expert who is exploring whether an idea is worth pursuing at all. A person who has never wired a board cannot sit down in front of an industrial simulator and get useful work done in an afternoon.

There is a second answer that has become tempting in the last few years, which is to ask a large language model to write the control code. This works until it does not, and the way it fails is dangerous in this setting. A language model produces plausible text, and plausible control code that calls a sensor the robot does not have, or that assumes a capability the hardware lacks, reads exactly like correct control code to a beginner. The output looks authoritative and is sometimes wrong, and the person least able to tell the difference is the very person the tool is meant to help.

There is a third failure that the first two share, and it is the one Kodro is named against. A simulator that reports a tidy number without saying how much to trust it invites the same misplaced confidence as a hallucinating model. A pretty run that quietly runs a slow robot faster than it can really go, or reports a top speed it never actually simulates, is a lie told in the language of engineering. The response is not to hide the approximations but to badge them: to say plainly, figure by figure, which numbers the simulation honours, which it approximates, and which it merely reports without ever driving.

Kodro is built where these observations meet. It is a proving ground: a place where a skeptical builder imports a real robot specification and sees how that machine would perform, on one ordinary laptop, offline, with no account and no cloud subscription, so cost and privacy are not barriers and a locked-down or disconnected machine is not an obstacle. It treats the language model as a useful but untrusted assistant rather than an oracle, grounding every suggestion in the robot the user actually built and placing a deterministic simulation gate between any generated program and the world. And it treats its own physics the same way it treats the model, as useful but not to be trusted blindly, disclosing the fidelity of every figure it reports rather than presenting a first-order model as a certified result. The same platform still serves the learner

who has only an idea and no spec, through a parts catalogue and a design studio, so the proving ground and the teaching studio are two faces of one tool.

1.2 Problem statement

The problem this project addresses can be stated in one sentence. How can a single, offline, free desktop application let a builder import a real robot specification, or a non-expert design one from a catalogue, and validate that robot's behaviour in a realistic, agent-populated simulation before building any hardware, with every reported figure carried at an honestly stated level of fidelity, and with a local language model for help that is genuinely useful and genuinely safe?

Four words in that sentence carry most of the weight. *Realistic* means the simulation has to be worth trusting: the robot must behave in a way that reflects its build, the world must contain things that move and respond, and a run must report what happened across varied conditions rather than one tidy outcome. *Honest* means the tool must not overclaim its own physics: every figure it reports is labelled as honoured, approximated, or not simulated, so a builder knows exactly how much weight each number will bear. *Offline* means a hard constraint, not a preference: no cloud service, no account, no paid interface, no data leaving the machine, which rules out the easy path of calling a large hosted model and forces every capability onto local resources. *Safe* means that the help the tool gives cannot quietly produce a program that would drive a real machine into a person, which makes the relationship between the model and the validator the central design question rather than an afterthought.

1.3 Aims and objectives

The aim of the project is a working offline proving ground on which a builder imports a real robot specification, or a non-expert designs one, and validates that behaviour in a realistic, agent-populated simulation before building it, with every reported figure carried at an honestly stated level of fidelity. Each objective below carries a plain test of done, so progress is judged rather than asserted.

1.3.1 Core objectives

These define the artefact. The platform must achieve all of them to count as a working system.

O1. **Specification-driven behaviour, imported or designed.** Let the user import a real robot specification of measured numbers, or assemble a build from a parts catalogue, where the specification drives the simulation. *Done when* changing the build changes behaviour: a heavier robot accelerates more slowly and drains its battery faster, a stronger motor raises the closed-form top speed, and a fitted sensor unlocks its command while an absent one withholds it from every way of programming the robot.

O2. **Honest fidelity disclosure.** Report every performance figure at one of three disclosed levels, honoured, approximated, or not simulated, and never present an approximation as a certified result. *Done when* each figure carries a fidelity badge in the Lab, the realism dashboard and a per-robot verification report, and a value the simulation does not actually drive is never badged honoured.

O3. **Safe execution and scoring.** Execute the robot's real program in a sandbox that blocks file, network and process access, and score every run against the scenario's success criteria. *Done when* a hostile program cannot reach the disk and a correct one is scored correctly.

O4. **Realistic, randomised validation.** Run the robot in a world with terrain and moving agents, across repeated randomised runs. *Done when* one program runs over several seeds and

returns a report of successes, collisions, time to goal and battery used.

O5. **Grounded local assistance with a guaranteed fallback.** Run the assistant on a local model only, grounded in the robot's specification, and fall back to deterministic rule-based behaviour within a fixed time budget when no model is present. *Done when* the platform is fully usable with the network off and no model installed, and when the model cannot call a part the build lacks.

O6. **Refinement from use without retraining.** Refine suggestions from accumulated use through reflection retrieval and a reusable skill library. *Done when*, across a session, the system reuses a stored reflection or skill and the median iterations to a working design fall.

1.3.2 Secondary objectives

These raise the quality of the artefact once the core holds.

O7. **Two ways to program, one meaning.** Provide a text editor and a blocks palette that compiles to the same program, so that the way the user expresses a behaviour does not change what the behaviour means.

O8. **One shared motion model.** Drive the simulation from a single set of closed-form equations and constants, mirrored across the two engines and locked together in continuous integration, so the visible studio and the tested Python engine cannot silently disagree about how a robot moves. *Done when* a constants hash, a golden-trace corpus and a formula-parity test all gate the two engines on every push.

O9. **Legible realism on a normal laptop.** Render the worlds, the named mission sites and the robot with enough fidelity to read as believable in a screenshot, while staying smooth on a machine with no discrete graphics card.

O10. **An honest record.** Build the system as small, independently tested changes behind a continuous integration gate, and keep a decision log and a test-evidence log, so the work is reproducible and the dissertation can be checked against it.

1.3.3 Out of scope

Two things are deliberately not attempted, and saying so keeps the claims honest. Kodro does not aim at production-grade physical accuracy; its motion is a believable kinematic model, not a rigid-body solver, and the discussion chapter is explicit about what that does and does not buy. Kodro also does not aim to replace the established heavyweight simulators such as Isaac Sim, Gazebo, Webots or MuJoCo; the useful relationship with those tools is a research bridge on the roadmap, not a claim of equivalence.

1.4 Contributions

This project contributes a complete, working artefact and the engineering record behind it. Concretely it delivers:

- a desktop application, offline by default, that imports a real robot specification of measured numbers, or lets a non-expert assemble a build from a parts catalogue, and derives the robot's mass, closed-form top speed, battery life and the exact set of commands it can run;
- a three-tier fidelity disclosure, honoured, approximated and not simulated, surfaced as per-figure badges in the Lab, in a realism dashboard and in a per-robot verification report a builder can keep, so the tool never dresses an approximation as a certified result;
- one shared motion model, a single set of closed-form equations and constants mirrored in JavaScript and Python and locked together by a constants hash, a golden-trace corpus and a

formula-parity test, so the visible studio and the tested engine cannot silently drift;
- two ways to program one robot, a Python subset interpreter and a blocks palette that compiles to the same Python, both driving a single command surface so the meaning of a behaviour does not depend on how it was written;
- a grounded local assistant and an automatic code reviewer that suggest and check code only against the parts the robot carries, backed by a deterministic rule engine that takes over within a fixed time budget when no model is installed, so the model can help but never has the last word on safety;
- a validation layer that runs a program in a sandboxed world of seventeen named mission sites among moving agents and reports what happened, with a self-refinement memory of reflections and reusable skills that informs later sessions without retraining any model;
- a grounding metric and a seeded benchmark harness, KodroBench, that measure by syntax-tree analysis whether a generated program stays inside the build's fitted-command set, with a leaderboard generated from a machine-readable run record rather than typed by hand; and
- a disciplined development process, recorded in a decision log and a running test-evidence log and gated by continuous integration, that makes the work reproducible and gives the critical reflection in this dissertation a factual basis.

The contribution is not a new algorithm. Retrieval grounding, a sandboxed gate, an experience memory, a skill library and domain randomisation are each known. The contribution is the demonstration that a real specification import, a disciplined three-tier fidelity disclosure and a single shared motion model combine with these known parts into an honest proving ground that runs entirely on one machine, and a measured account of how far offline self-refinement can go when changing the model weights is off the table.

1.5 Report structure

The remainder of this dissertation is organised as follows. Chapter 2 reviews the literature the design follows from, across learning by building, the gap between simulation and reality, and the grounding and honest framing of language models. Chapter 3 sets out the requirements, the intended users including who the tool is not for, and the analysis that shaped the design. Chapter 4 describes the layered architecture and the key design decisions, chief among them the robot specification as the single source of truth and the deterministic gate that stands between a generated program and the world. Chapter 5 describes the implementation against the real code, the techniques that recur, and the engineering challenges that were met and fixed. Chapter 6 reports the evaluation, separating what was measured from what is planned, and stating the threats to validity plainly. Chapter 7 discusses the honest limitations of the system. Chapter 8 concludes and sets out the future work. The references and appendices follow.

Chapter 2

Background and Related Work

This chapter situates Kodro against three lines of work rather than placing it beside them. The design is a consequence of these lines, so the chapter reads each one for what it implies for an offline design and validation platform, and ends by naming the niche the surveyed work leaves open.

2.1 Learning by building

The oldest of the three lines is constructionism, the durable finding that people learn most deeply when they build something external that they can watch behave and then debug (Papert, 1980). The important word is *watch*. A learner who can see the consequence of a change, and can change it again, is in a tight feedback loop that abstract instruction cannot match. Kodro is built around exactly this loop. The user assembles a robot, writes a behaviour, runs it, sees the robot behave in a world, and adjusts. The blocks palette compiles to the same Python the text editor runs, so the move from blocks to text is a change of surface and not a change of tool, and the learner is never asked to throw away what they understood in order to progress. This is a deliberate echo of the constructionist position that the medium should not get in the way of the idea.

2.2 Trusting a simulator: the gap between simulation and reality

The second line is the question of why a simulator is worth trusting at all. The central and well-documented problem in robotics is the gap between simulation and reality, the drop in performance when a behaviour tuned in simulation meets the physical world. The accepted response is not the pursuit of a perfect simulator, which is unattainable, but domain randomisation: vary the friction, the mass, the sensor noise and the scene across many runs, so that a behaviour which survives the spread is likely to survive reality too (Tobin et al., 2017). The insight is that variability is the asset, not the enemy. A program that only works on one tidy layout has learned the layout; a program that works across a randomised spread has learned something more general.

Kodro adopts this directly. A design is validated across randomised seeds with varied friction, mass and sensor noise, and a run reports the spread and not only the average. The worlds are populated with agents that move and respond, traffic that keeps to one-way lanes and brakes for the robot, pedestrians that use the crossing, so that a self-driving car or a home robot is tested among things that move rather than on an empty plane. The design follows the literature here because the literature is right: the value of a simulator is not its prettiness but the honesty of the spread it reports.

2.3 The model: hallucination, grounding, and honest refinement

The third line concerns the language model itself, and it justifies both the offline choice and the careful framing of refinement. Two findings shape the design. The first is that hallucination is intrinsic to these models rather than a defect that a future version will simply remove (Huang et al., 2023). In generated code this surfaces as plausible but wrong calls (Lee et al., 2025), which is precisely the dangerous failure when the output is control code that a beginner cannot evaluate. The second is the standard mitigation: ground generation in retrieved, authoritative

material rather than in the model's memory (Lewis et al., 2020). For robots, the strongest form of grounding is to constrain the model to compose a known, safe set of capabilities rather than to invent them, in the spirit of programs written against a fixed robot interface (Liang et al., 2023; dos Santos et al., 2026) and of grounding instructions in what the robot can actually do (Ahn et al., 2022).

Kodro grounds every suggestion in the robot the user assembled, so the model cannot call a sensor that is not fitted. This grounding is also why no cloud is needed: the material the model is grounded in, the build, lives on the machine, so the help can be local. On refinement over time the design is deliberately careful, because the honest state of the art is that genuine weight-level self-evolution is unsolved (Tao et al., 2024). Kodro therefore does not claim it. Instead it refines at the level of the system, through a verbal reflection loop (Shinn et al., 2023) and a growing library of behaviours that worked (Wang et al., 2023), composed for new designs without retraining, mirroring recent experience-driven self-evolving agents that improve from their own history without changing weights (Wu et al., 2025). Where several agents cooperate, naive arrangements fail in characteristic ways (Cemri et al., 2025), which is why in Kodro a deterministic check, not another model, always has the final word. Before that check an automatic reviewer flags suspect code, following recent automated code-review practice (Sun et al., 2025), and deterministic static analysis can catch hallucinated code outright (Khati et al., 2026).

2.4 Small local models

A fourth, narrower body of recent work makes the offline choice practical rather than merely principled. Small open-weight models, in the one to three billion parameter range, paired with retrieval, now give reliable and low-hallucination help offline (Reza et al., 2025), and onboard small models are being shown to drive robot task planning and code generation on constrained hardware (Wang et al., 2026). This matters because the size is the binding constraint for the target machine. A model in this range is the largest class that still answers at an interactive pace on a mainstream laptop with no discrete graphics card, where a seven billion model would demand a card a school or home laptop rarely has. The honest limit, discussed again in the evaluation and the conclusion, is that a small local model buys domain grounding and a reliable output format, not frontier capability, so the design leans on grounding and the simulation gate and never on the model being clever.

2.5 The niche

The surveyed work leaves a specific gap open. There are excellent heavyweight simulators for experts, excellent block-based environments for children, and a growing literature on grounding and on honest self-evolution, but there is no single offline desktop tool that lets a builder import a real robot specification, or a non-expert design one, program it two ways against one meaning, and validate its behaviour among moving agents across a randomised spread, with every reported figure carried at an honestly stated level of fidelity, a local model that is grounded hard enough to be safe, and a deterministic gate that has the final word. The nearest evaluation harness in the robot-programming literature, RoboEval with its CodeBotler programs (Hu et al., 2023), checks generated code against one fixed robot interface, so the failure it can surface is an invalid argument to a valid call, whereas the grounding failure that matters here is an invented symbol against an interface that changes with every build. Kodro is built to fill exactly that gap, and the rest of this dissertation describes how, and reports how far it gets.

A structured survey of the competitive landscape sharpens where that gap sits. The professional simulators, Isaac Sim, Gazebo, Webots, CoppeliaSim, MuJoCo and PyBullet, are powerful and

mostly open source, but they assume an expert who can author a robot description by hand, several require a specific GPU or a vendor account, and none present a non-expert with a stated fidelity for the numbers they produce. The education platforms, from VEXcode VR to CoderZ to Open Roberta, are approachable but ship a fixed preset robot with no way to import a custom parts specification, are usually cloud hosted behind an account, and several state plainly that they ignore friction or physics by design. The parts calculators that do predict real performance, from eCalc to the FRC drivetrain spreadsheets, are single purpose tools with no simulation world, no code, and no assistant. No surveyed product combines an offline desktop application, no account, a custom parts to behaviour import, an honestly disclosed fidelity for every figure, and an AI assistant behind a deterministic safety gate. That specific combination is the differentiator, and it is what the rest of this dissertation builds and measures. The assistant is offline by default on a local model, and a user may optionally connect their own cloud key for a stronger model without weakening the offline guarantee.

Chapter 3

Requirements and Analysis

This chapter turns the aims of Chapter 1 into requirements that the design can be measured against. It begins with the intended user, because the user decides almost everything that follows, and it is deliberate about who the tool is not for. It then states the functional and non-functional requirements, and analyses the central tension that shaped the whole system, which is the relationship between a helpful model and a trustworthy result.

3.1 The intended user, and who it is not for

Kodro is built for two related people, and the proving-ground framing sharpens both. The first is the skeptical builder: someone who has a real robot in mind, or on paper, and wants to know how it would actually perform before committing money and hardware, and who will not be satisfied by a pretty animation that hides its assumptions. That person imports a specification of measured numbers and reads the fidelity-badged report the way an engineer reads a datasheet, trusting the honoured figures, weighing the approximated ones, and noting what is not simulated at all. The second is the capable non-expert who has only an idea: a person who can think clearly and follow a structured tool but does not have a lab, a kit, a budget for parts, or training in robotics. This is the maker weighing a companion-robot idea, the student testing a self-driving behaviour without a car, the hobbyist checking whether a rover handles rough ground before buying it. That person assembles a build from the catalogue and gets from an idea to a validated behaviour in an afternoon, on the laptop they already own, without spending anything and without an account. The tool earns its place with both by being honest: it tells the builder exactly how much to trust each number, and it never lets the learner ship a program it has not actually checked.

It is equally important to say who the tool is not for, because a tool that claims everyone as its user serves no one. Kodro is not for a professional roboticist refining a known design to deployment tolerances; that person needs rigid-body physics and the established heavyweight simulators, and Kodro does not pretend to compete with them. It is not a children's coding toy; the framing, the language and the expectations assume an adult who wants a serious result, not a gamified first lesson. It is not for a team that needs to collaborate over a network on a shared design; the offline, single-machine constraint that makes Kodro safe and private also makes it deliberately solitary. And it is not for anyone who needs the model to be brilliant, because the model is small, local and grounded on purpose, and the design never asks it to be clever. Naming these non-users is not modesty. It is the discipline that keeps the rest of the requirements honest.

3.2 Functional requirements

The functional requirements follow the core objectives. They are stated as testable capabilities.

3.3 Non-functional requirements

The non-functional requirements are where the project's character lives, because the offline constraint touches every one of them.

- **Offline by default.** No account, no paid interface, and no data leaving the machine on any

default path: the only networked call the core makes is to a local model server on the loopback address, and the platform is fully functional with that absent. A user may explicitly opt in to connect their own cloud model key for a stronger assistant, in which case only their prompt is sent, and only to the provider they choose; the offline default is never weakened and requires no such connection.

- **Runs on a mainstream laptop.** The worlds and the robot must render smoothly on a machine with no discrete graphics card, which bounds the visual approach to procedural geometry and a measured lighting model rather than heavyweight assets.
- **Deterministic where it matters.** The simulation, the scoring and the safety checks must be reproducible, so they can be tested without a display and so a result can be trusted to mean the same thing twice.
- **Safe by default.** The system must fail toward safety. An absent model, a missing audio device, a headless environment, or a hostile program must each produce a clear, contained outcome rather than a crash or an unguarded action.
- **Legible.** A non-expert must be able to read the program surface and understand it, which is why the command API is a small set of plainly named functions rather than an object hierarchy.

3.4 The central analysis: a helpful model against a trustworthy result

The hardest requirement to satisfy is not any single item in the tables. It is the tension between two of them. Requirement F8 wants the model to be helpful, and helpfulness pushes toward letting the model write whatever code seems to fit. Requirement F5 and the safety non-functional want the result to be trustworthy, and trust pushes toward letting nothing run that has not been checked. A naive design resolves this tension in the model's favour and ships plausible but wrong control code to a beginner. A timid design resolves it in the validator's favour and produces a tool too rigid to help.

The analysis that shaped Kodro refuses to resolve the tension in either direction and instead separates the two concerns by authority. The model is allowed to be helpful, to draft, to suggest, to explain, but it is never allowed to be the thing that decides a program is safe to run. That decision belongs to deterministic machinery: the grounding that will not let a suggestion name a missing part, the sandbox that blocks unsafe operations before execution, the automatic reviewer that flags suspect code, and the simulation gate that runs the program in a world and scores it before any hardware is involved. The model proposes; the deterministic layer disposes. This single principle, that the model may generate but the validators have final authority, is the spine of the architecture in the next chapter, and the evaluation returns to it to ask whether it actually holds in practice.

ID	Requirement	Test of satisfaction
F1	Import or assemble a robot	The user imports a real specification of measured numbers, or picks a chassis, board, sensors and actuators, and the build derives mass, top speed, battery capacity and the set of runnable commands.
F2	Specification drives behaviour	Changing the build changes the simulation: a heavier robot accelerates more slowly and drains battery faster; a stronger motor lifts the closed-form top speed.
F3	Fidelity disclosure	Every performance figure carries a fidelity tier of honoured, approximated or not simulated, in the Lab, the realism dashboard and a per-robot verification report; a value the sim does not drive is never badged honoured.
F4	Command gating	A command appears only if the part it needs is fitted, and disappears from the editor, the blocks palette and the assistant when the part is removed.
F5	Two ways to program	A text editor and a blocks palette that compiles to the same program, both driving one command surface.
F6	Safe execution	A program runs in a sandbox that blocks file, network and process access, and a hostile program cannot reach the disk.
F7	Scenario scoring	A run is scored against the scenario success criteria, returning a pass or fail, a score, and a per-criterion reason.
F8	Randomised validation	One program runs across several seeds with varied friction, mass and sensor noise, returning a report of successes, collisions, time and battery.
F9	Grounded assistant	The local model suggests code only against fitted parts, and refuses and points back to the build when asked for a part that is absent.
F10	Deterministic fallback	With no model installed, a rule engine returns code and checks within a fixed time budget, and the platform stays fully usable.
F11	Self-refinement memory	After a run the system records a reflection and may save a skill, and retrieves similar past material to inform later suggestions.

Table 3.1. Functional requirements and their tests of satisfaction.

Chapter 4

Design and Architecture

This chapter sets out the design that the requirements imply. Every claim here corresponds to code in the repository, and the rationale for the non-obvious choices is recorded contemporaneously in the project's decision log. The chapter moves from the overall shape down to the parts that carry the most weight: the robot specification as the single source of truth, the trace-driven core that makes everything testable, the safety machinery, and the grounded assistant with its deterministic fallback.

4.1 Design goals

Four goals shaped the architecture, in priority order. First, **legibility**: a non-expert must be able to read the program surface and understand it. Second, **determinism and testability**: the simulation, scoring and safety checks must be reproducible and testable without a display. Third, **strict offline operation**: no network dependency anywhere on the critical path. Fourth, **separation of the user's program from the engine internals**, so the user works against a tiny, safe surface and the engine can change underneath without breaking their mental model.

4.2 Technology choices and their rationale

The platform is written in Python 3.12 behind a desktop web view, and each choice follows from the offline, low-resource constraint rather than from fashion. The interface is authored in React but precompiled to a single bundle, so the panelled layout and the modern look are kept while the cost of a runtime compiler is paid once at build time and never on the user's machine. The three-dimensional world uses a copy of the Three.js library held inside the project, core only, so the graphics work with the network off and with no asset pipeline. The user's designs and the system's accumulated record are stored in SQLite, a single standard-library file with no server, so a backup is a file copy and nothing leaves the machine, and a whole project, the build, the programs, the world and the accumulated memory, saves to and loads from a single self-contained `.kodro` file. The assistant calls a local model served by Ollama, which installs as one small application and keeps a model warm with a single command a non-technical user can follow. The desktop shell is pywebview, which renders the vendored interface in a native window using the platform web view, with a Tkinter interface retained as a guaranteed-offline fallback for machines without a modern web view runtime.

4.3 Layered architecture

The system is organised in layers with a one-directional dependency flow, so that each layer depends only on the ones below it. Figure 4.1 shows the design surface where this begins, the Robot Lab, and the layering is as follows.

4.4 The robot specification as the single source of truth

The most important design decision in Kodro is that the robot is not cosmetic. Its specification is a single object that the rest of the system reads, and it is the reason changing the build changes

Layer	What it is	Where it lives
Program surface	The Python subset and the blocks palette	`interpreter.js`, blocks palette
Command registry	The single set of commands the build allows	derived from the robot specification
Robot specification	The single source of truth, imported or designed	`RobotLab.jsx`, `specschema.js`, the derive step
Shared motion model	The closed forms and constants both engines obey	`motion-model.js`, `engine/motion_model.py`
Fidelity disclosure	The three-tier badges and verification report	`specschema.js`, `realism.jsx`, `verify.jsx`
Worlds and motion	Worlds, named mission sites, the natural-motion tick	`Viewport3D.jsx`, `terrains.jsx`, `agents.jsx`
Validation and memory	Scoring, randomised runs, reflections, skills	`memory.jsx`, the Python grader
Python engine	The metres world, physics wrapper, public API	`engine/`, `rover_api.py`

Table 4.1. The layered architecture and where each layer lives in the codebase.

the behaviour. The specification arrives one of two ways. A non-expert assembles it in the Robot Lab, where a derive step computes from the chosen parts a structured build: the chassis type, the board, the fitted sensors and actuators, and from those a mass in kilograms, a top speed, an acceleration, a turn rate, a battery capacity and a drain rate, a friction profile, and crucially a list of supported commands. A builder with a real machine instead imports a Kodro Robot Specification, a small JSON document validated by a schema in `specschema.js`. It carries the total mass, the body footprint in centimetres, the drive geometry (wheel radius, wheelbase, motor count and per-motor no-load rpm, stall torque and nominal voltage), the battery capacity, voltage and usable fraction, and the sensors with their mount position, yaw and range. The schema drops unknown keys with a warning, clamps mildly out-of-range values visibly rather than silently, and rejects wild ones, and it round-trips: a build can be exported back out as a specification for a peer to import. From that point the imported numbers, not a catalogue proxy, drive the simulation.

The command list is what makes the design honest. The command registry is generated from the specification, not hand-written per panel. A build without an ultrasonic sensor has no distance-reading command in the registry, so the text editor will not accept it, the blocks palette does not offer it, and the grounded assistant cannot suggest it. There is no second place where commands are defined and could drift out of step, because there is only one registry and every surface reads it. This is the structural guarantee behind the command-gating requirement, and it is the mechanism by which the model is grounded: the model is told the registry, and the registry is exactly what the build allows.

The physical fields drive the motion. For an imported build the top speed follows a closed form from the motor rpm and wheel radius, the tractive force from the stall torque and motor count, the acceleration and braking from that force against the mass, and the battery runtime from the capacity against the drawn power; for a catalogue build the same character is derived from the chosen parts. Either way a heavier robot has a lower acceleration and a faster battery drain, a stronger motor raises the top speed, and the surface traction changes how the robot holds the ground. The point is that these are not separate sliders the user tunes; they fall out of the

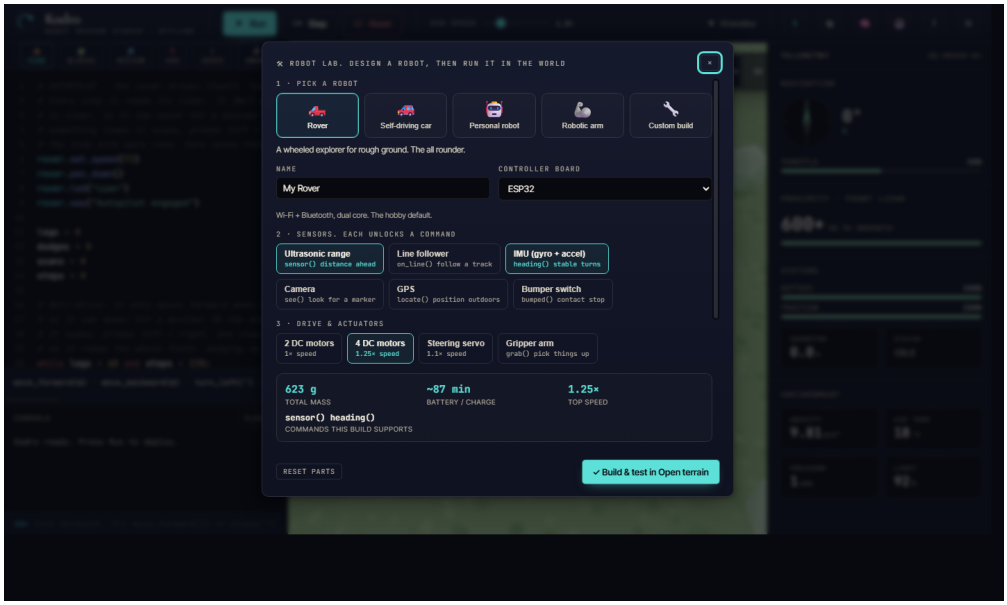

Figure 4.1. The Robot Lab. The user assembles a robot from a board, sensors and actuators, and the build's mass, battery life, top speed and supported commands update live. That specification then drives the simulation.

specification, so the design loop is to change the robot, not to change a number, and to watch the behaviour change as a consequence.

4.5 The three-tier fidelity disclosure

A proving ground earns trust by being candid about its own limits, so the second load-bearing design decision is that every figure the simulation reports carries a disclosed level of fidelity. Three tiers do the work. A figure is **honoured** when the simulation actually runs the measured number: the commanded distances and turn angles, the collision circle sized from the body footprint, the command availability gated on fitted parts, the battery treated as a hard budget the robot halts at, and, when a build's motor-derived top speed falls inside the simulable band, that top speed itself. A figure is **approximated** when a first-order model stands in for the real physics: acceleration and braking as a trapezoid rather than an integration of forces, turn timing from track geometry rather than wheel torque curves, traction as three coarse bands per surface, and battery drain as a constant-power ledger with no voltage sag or thermal derating. A figure is **not simulated** when it is reported for reference but never actually driven: slopes and terrain height, wheel-level slip and per-motor torque curves, suspension and three-dimensional contact, and the command semantics of the parts that carry no runnable command yet.

The discipline of this scheme is that it is enforced at the edges, not asserted in prose. A top speed that falls outside the band the simulation can run is a case worth dwelling on, because it is exactly where an honest tool is tempted to cheat. A cheap educational rover with a slow motor has a real top speed below the simulable floor; rather than run it at its true crawl, the simulation runs it at the floor, which is faster than real. The honest response, and the one the code takes, is to refuse to badge that number honoured: the badge drops to approximated and the warning states the real direction, that the top speed is below the floor and is simulated faster, rather than papering over the substitution. The same holds at the ceiling for a fast build. This case matters because it is the precise failure the whole product exists to prevent, a distorted number shown under a reassuring badge, and the design closes it by making the badge follow the truth of what the simulation runs. The tiers are surfaced three ways: as per-figure badges beside the live

specification in the Lab, as a full table in a realism dashboard, and as a per-robot verification report, a standalone document that lays out the build as simulated with each figure tagged by tier and by how it was derived, for a skeptical builder to keep and check.

4.6 One shared motion model

The third design decision answers a question the first two raise. If the imported specification drives the simulation, and the fidelity badges promise that a figure is honoured, then the number the user sees in the visible studio and the number a test asserts in the Python engine must be the same number, or the honesty is a fiction. Early in the project they were not the same. Four hand-rolled kinematic replicas, one in the JavaScript studio, one in the Python engine, one in the scenario validator and one in the self-test, drifted apart until they disagreed by an order of magnitude on how much battery a metre of travel costs. The design response was to collapse them into one shared motion model: a single set of closed-form equations and named constants, expressed once and mirrored in two files, `motion-model.js` for the studio and `engine/motion_model.py` for the Python engine.

The model is locked at three levels, each a gate in continuous integration. The constant table is hash-gated: both files serialise their constants to a canonical form and a test fails if the two hashes differ, so a constant cannot be changed in one engine without the other. The catalogue-mode behaviour is golden-trace-gated: a corpus of programs is run through both engines and their displacement, distance, heading and battery are asserted to agree, so a change that alters how a robot moves fails unless both engines change together. And, most recently, the physical-mode closed forms are formula-parity-gated: the twelve closed forms plus the sensor-pose transform were ported from the studio's JavaScript into the Python engine, and a test runs the studio model over a reference robot and fails on any drift. An imported build's derived numbers, its top speed, stall force, mobility, acceleration, energy, runtime, slope, turn timing, stopping distance and sensor pose, are therefore provably identical across the two engines. The honest scope of this parity is stated plainly in the limitations chapter: the two engines share the formulas, but the full end-to-end simulation of an imported build, in particular the sensor rays that account for a sensor's mount pose and range, runs only in the studio, while the Python engine reproduces the derived numbers and grades catalogue-mode motion. The design goal was never to claim identical simulators; it was to make the numbers the fidelity badges rest on impossible to fake in one engine and not the other.

4.7 The trace-driven core

A single abstraction underpins scoring, hinting and replay: the trace. Every call the running program makes to the command surface appends an event that records the call, its arguments, its result, a snapshot of the robot's state, and any collision. From this one trace three features are derived without duplicating the representation of what the robot did. Scoring compares aggregates of the trace, the samples collected, the collisions, the battery used, the distance, the final position, against the scenario's declared success criteria, and an analysis of the source against the allowed constructs, to yield a pass or fail, a score, and a per-criterion reason. Hinting runs a set of rule predicates over the same events plus the score and surfaces the first that matches. Replay reconstructs the run from the serialised trace and scrubs through it.

The value of this choice is that the decision logic is pure. The scorer is a function from a scenario, a trace and the source to a result, with no input or output and no hidden state, which makes it trivially testable with synthetic traces and impossible to make non-deterministic by accident. The same purity holds for the hint rules and the report. Side-effecting wrappers, the file writes and

the dialogs, are thin shells over these pure cores. This is why the system can be tested thoroughly without a display, which the evaluation chapter relies on.

4.8 Safety: the sandbox, the executor, and the gate

The user's program is untrusted, and the design treats it that way at three points. Before execution, the sandbox walks the program's syntax tree and rejects unsafe imports and builtins, the file opens, the process spawns, the attribute walks that reach for forbidden capabilities. During execution, the executor runs the program in a daemon thread under a hard wall-clock timeout, folding every failure mode, a syntax error, a runtime error, a sandbox violation, a timeout, into one structured result rather than letting an exception escape. After execution, the simulation gate is the run itself: the program drives the robot in a world and is scored, so a wrong program fails visibly on screen against the scenario rather than silently on real hardware.

These three points are the concrete form of the central analysis from Chapter 3. The model never touches any of them. It cannot relax the sandbox, it cannot extend the timeout, and it cannot decide that a program passed. The deterministic layer owns safety end to end, which is what lets the model be helpful without being dangerous.

4.9 Worlds, mission sites, and moving agents

The validation has to happen somewhere believable, so the design recommends a world to fit the robot. A self-driving car is dropped into the City, with one-way lane traffic that loops and brakes for the robot and pedestrians that keep to the pavements and use the crossing. A home or arm robot is dropped into the Room, with furniture and people. A rover is dropped onto a planetary terrain. Beyond these base worlds sit seventeen named mission sites, real places given their own treatment: the Sahara, the Amazon, Antarctica, the Thar Desert in India, the Maasai Mara, the slopes of Mount Fuji, the Giza plateau, an Icelandic lava field, the Himalayan foothills, the Great Barrier Reef, the Mariana Trench, Olympus Mons on Mars, the Moon's Tycho crater, Europa's ice crust, a robotics test bay, a warehouse and a minimal debug grid. Each carries its own traction, lighting and atmosphere, so the same program behaves differently across them and a run reads as a place rather than a plane. The open worlds are alive: traffic keeps to one-way lanes and eases rather than stepping, pedestrians walk their pavements and a few cross at the crossing, and a small autonomous fleet of other robots roams the open terrains, picking fresh goals and steering around the obstacles, each other and the user's robot, so the scene has machines to share it with rather than being an empty stage. Figure 4.2 shows the City world in three dimensions, which the user can orbit a full turn to read the scene from any angle.

The motion is designed to read as natural rather than to be physically exact. The robot does not slide like a prop; it transfers weight under acceleration, banks into turns, settles on its suspension, and steers from the front wheels where the chassis has them. The honest description of this, returned to in Chapter 7, is that it is a per-type kinematic model that fakes the feel convincingly rather than a rigid-body solver that resolves forces. This is a deliberate trade. The believable feel is what a non-expert needs to judge a behaviour, and it runs smoothly on a laptop, where a full solver would not.

4.10 The grounded assistant and its deterministic fallback

The assistant is designed around the principle that the model may generate but the validators decide. It is given the robot specification and the command registry as its grounding, and it is asked to suggest code only from that registry. When the user asks for a behaviour that needs a part the build lacks, the assistant is designed to refuse and to point back to the specification,

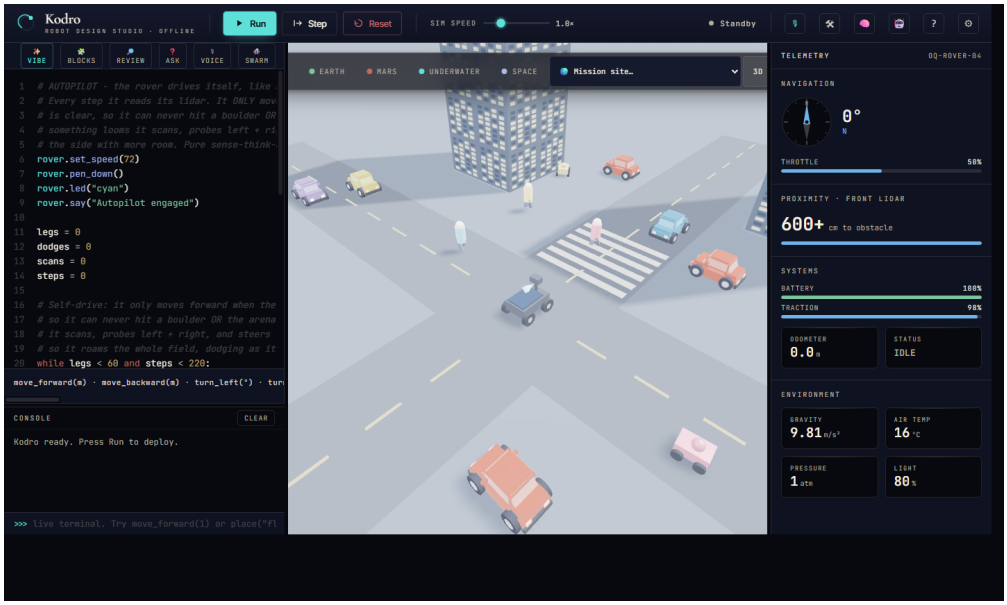

Figure 4.2. Validation in a realistic world. The designed robot runs in a city street rendered in three dimensions, among traffic and pedestrians that move around it, so its behaviour can be watched before any hardware is built.

explaining which part is missing, rather than to invent a call. An automatic code reviewer sits alongside it, proposing and then critiquing the model's output and accepting a rewrite only after it passes the same sandbox a manual run would.

Behind all of this is the deterministic fallback, which is what makes the offline guarantee real. If no model is installed, or if the model does not answer within a fixed time budget, a rule engine returns rule-based code and checks, and the platform stays fully usable. The fallback is not a degraded mode bolted on; it is the floor the whole assistant stands on, and the design treats the model as an optional accelerator above a guaranteed deterministic base.

4.11 Self-refinement without retraining

The final design element is the refinement loop, and the design is careful to call it refinement and not improvement, because nothing here changes the model's weights. Three local, countable mechanisms carry it, all in the local store. After each run the system writes a short reflection on what failed and what worked, and retrieves the most similar past reflections into the next suggestion. Routines that passed validation are kept as named, parameterised skills and composed into later designs. A record of past designs and their outcomes is sampled so that advice on a new build is anchored in what happened on similar ones. None of this edits the model; all of it is system-level memory, honestly framed, and the evaluation asks directly whether it produces the fall in iterations the objective requires.

Chapter 5

Implementation

This chapter describes how the design was realised. It is grounded in the codebase and in the running test-evidence log, and the engineering challenges are drawn from the real commit history, which makes them honest material for critical reflection rather than a tidy story told after the fact.

5.1 The program surface and the interpreter

The user's program runs through a Python subset interpreter, written in JavaScript and vendored into the bundle, which compiles a source string into a generator that yields motion and sensor events one at a time. The generator design is what lets the studio animate a run smoothly: the application pumps the generator, advancing the robot one event per tick, so the user sees the robot move rather than jumping to a final state. The interpreter supports the constructs a non-expert needs, sequence, selection, iteration with both `for` and `while`, functions, and the sensor reads, and it guards the pathological cases a beginner can stumble into, such as an enormous exponent, so that a slip produces a contained result rather than a hang.

A subtlety worth recording is that two call styles are both valid and both correct. A bare call such as `move_forward(2)` moves two metres, scaled internally to engine units, while a method call such as `rover.forward(200)` moves two hundred centimetres. This dual surface is convenient but it is also a known source of the arena-scale mismatch discussed in Chapter 7, and it is recorded honestly rather than hidden.

5.2 The robot specification and the derive step

The Robot Lab implements the single source of truth from the design. A catalogue of boards, sensors, actuators and chassis types backs a derive step that computes the structured specification from the chosen parts. The derive step is where the physical character of the robot is set: the mass is summed from the parts, the top speed and acceleration follow from the motor and the mass, the battery capacity and drain follow from the board and the load, and the supported-commands list is assembled from exactly the parts that provide each command. Saving a build dispatches an event carrying the recommended world, which the studio listens for, so designing a robot and being dropped into the right world to test it are one action.

5.3 Importing a real robot: the KRS schema

Alongside the catalogue sits the import path, which is what makes Kodro a proving ground rather than only a studio. The Kodro Robot Specification is a JSON document with a schema and a validator in `specschema.js`. A builder presses Import spec, the desktop bridge or a file dialog reads the document, and the validator returns a cleaned specification with a list of warnings and errors surfaced in the Lab. The validation is deliberately forgiving where it can be and strict where it must be: an unknown key is dropped with a warning rather than rejected, a value mildly outside its band is clamped and the clamp is shown, and a value wildly out of range is refused outright, so a hand-written or third-party specification imports usefully without silently becoming a different robot. When a specification carries motor and drive geometry, the physical

closed forms take over from the catalogue proxies: the derived object carries a `phys` block whose numbers, not a parts factor, drive the simulation. The same schema exports, so a build assembled in the Lab or imported and adjusted can be written back out as a specification for a peer, and the export carries the derived numbers alongside the inputs so the receiving side can check them.

5.4 The shared motion model and its conformance gates

The one shared motion model of the design is two mirrored files. `motion-model.js` holds the constants and the closed forms for the studio; `engine/motion_model.py` holds the twin for the Python engine. The physical closed forms are the load-bearing part: the top speed as $v = (\text{rpm}/60)\,2\pi r$, the tractive force as motor count times stall torque over wheel radius, the acceleration as that force less rolling resistance over mass, the battery drain from a rolling-resistance power model against the usable energy, the differential turn timing from track geometry and the Ackermann turn radius from wheelbase and steering angle, the stopping distance as $v^2/(2\mu g)$, and the sensor-mount transform that places a sensor's reading at its offset and yaw. Three tests gate the pair on every push. A conformance test serialises both constant tables to a canonical form and compares their SHA-256 hashes, so a constant edited in one engine and not the other fails the build. A golden-trace test runs a corpus of programs through both engines and asserts their displacement, distance, heading and battery agree, so catalogue-mode motion cannot drift. And a physical-golden-trace test runs the studio's closed forms over a reference robot through a small Node fixture and asserts the Python twin returns identical values to a relative tolerance of 10^{-12}, so the derived numbers an imported build shows are provably the same on both sides. The honest boundary, recorded in the limitations chapter, is that the Python engine's sensor rays still originate at the rover centre with fixed module ranges and do not yet consume an imported sensor's mount pose or range, and the public Python API does not import a specification's mass, rpm or battery, so a measured build is simulated end to end only in the studio while the Python engine mirrors the formulas and grades catalogue-mode motion.

5.5 The verification report

The fidelity disclosure is implemented as pure functions in `verify.jsx` that turn the active build into a prediction sheet and join it with the evidence the app already holds, the last live run's odometer and wall-clock and the last multi-seed validation report. Every row carries its figure, its value, its fidelity tier and a one-line statement of how it was derived, for example that a top speed came from $v = (\text{rpm}/60)\,2\pi r$ on the imported motor rather than from a catalogue anchor. The report renders to a standalone HTML file a builder can save and keep, and because the functions are plain JavaScript with no React they are exercised headless by the QA harness. Nothing in the report computes new simulation state; every number is either a motion-model closed form or a recorded result, which is what lets the report make its honesty claim truthfully.

5.6 The worlds and the natural-motion tick

The three-dimensional 3D Viewport is built on core Three.js with no asset loader, so every robot, obstacle, terrain feature and agent is procedural geometry generated in code, boxes, cylinders, spheres, extruded shapes and canvas textures, lit by an environment map with shadow mapping and tone mapping. The open-terrain ground is shaded rather than flat: a tileable normal map, derived by a Sobel pass over a periodic height field, and a roughness map are both generated in canvas at scene-build time, so the sun and fill light graze real micro-relief instead of a coloured plane. The City and the Room are built in code, and the moving agents, the traffic and the pedestrians, run in a shared coordinate space with one-way lanes, looping paths, and a rule that

brakes them for the robot. The natural-motion tick is the code that gives each robot type its feel: the per-type pitch under acceleration, the roll into a turn, the suspension settle, and the front-wheel steer, applied as a function of the robot's velocity and heading change each frame. The world is chosen from a catalogue of presets, the City and the Room alongside a Robotics Lab, a Warehouse, a Minimal Debug Grid and a set of real outdoor terrains, each carrying its own friction and lighting so the same program behaves differently across them. A render-quality control spanning Low to Cinematic bounds the two heaviest costs, the shadow resolution and the pixel ratio, so the scene stays smooth on a laptop with no discrete graphics card on the lower tiers and maxes its fidelity for a screenshot on the highest. The Cinematic tier additionally runs a small, hand-written, fully offline post chain, a bloom built from a bright-pass and a separable Gaussian blur composited additively over the base render together with a vignette, written directly against the core renderer because the library ships its post-processing only in an examples tree that is not vendored and cannot be fetched under the offline rule. The pass is gated so that it is disabled under a reduced-motion preference, turned off if the slow-hardware downgrade fires, and dropped to the plain render if the device cannot allocate its targets, so the offline and low-hardware guarantees hold on every tier. A first-person toggle drops the camera onto the robot for a driver view.

5.7 Offline shading without an asset pipeline

The offline rule shapes the renderer as sharply as it shapes the model. The single largest lever for visual quality in a real-time scene is usually an asset pipeline: a glTF or GLB loader and a set of authored, well shaded models. That lever is closed here. Three.js ships its loaders and its post-processing only in an examples tree that is not part of the vendored core, and the zero-network constraint means neither the loader nor any quality interchange model can be fetched and committed; a hand-authored model would in any case be no better than the procedural geometry it would replace. The honest design response is to push the procedural surfaces as far as they will go and to be candid that an importer is future work rather than a missing feature that was skipped.

Two pieces of that work are worth recording because they were done under the constraint rather than around it. The first is surface relief. The open-terrain ground already carried a baked colour grain, but light still struck it as if it were glass, which was the strongest tell that the world was a tech demo. Each ground now generates, in canvas at scene-build time, a tileable normal map together with a roughness map. The normal map is a Sobel pass over a periodic height field whose waves are integer multiples of the tile size, so the derived normals are seamless under repeat wrapping; the result is that the existing physically based sun and fill light graze real micro-relief, and sand reads as sand, regolith as pits, the seabed as ripples. No texture file is loaded and no new binary is vendored.

The second is a post-processing pass for the highest quality tier. Because the library's bloom pass is not available offline, the pass is written directly against the core renderer: the proven base image is drawn to the canvas exactly as the lower tiers draw it, the scene is then rendered once more into an offscreen target used only as a bloom source, and a bright-pass, a separable Gaussian blur, an additive bloom composite and a vignette are layered on top. Drawing the bloom additively over the unchanged base, rather than replacing the image, means the worst case is that the scene simply looks like the normal render, which makes the feature safe to ship. It is gated to the Cinematic tier, disabled under a reduced-motion preference, turned off automatically if the slow-hardware downgrade fires, and dropped to the plain render if the device cannot allocate its render targets or if the pass throws at frame time. The shading and the post pass are headless safe: with no document or WebGL context the generators return nothing and the scene renders

exactly as before, so the offline bundle-render test never touches a canvas. This is the offline constraint treated as a design input, not an apology.

5.8 Memory, the assistant, and the fallback

The self-refinement memory is implemented over the local store as reflections and saved skills, written after a run and retrieved before a suggestion. The assistant is implemented as a client to a local Ollama server, preferring a small coder model and grounding each request in the specification and the command registry, with an automatic reviewer that proposes and then critiques. The most important implementation detail is the guard: every networked or optional capability, the model, the audio, the achievement toasts, is best-effort and hard-guarded, so an absent server, a machine with no audio device, or a headless environment produces a silent no-op rather than an error. The deterministic rule engine sits underneath as the guaranteed path, and the switch to it is automatic on a missing model or a timeout.

5.9 Engineering discipline

Each unit of work closed only when a full gate passed: the test suite with branch coverage, the linter, the format check and a strict type check, after which the change was committed, pushed, and verified green on continuous integration. The coverage gate ratcheted upward as the codebase matured and settled at a minimum of eighty five percent line and branch, which every push must clear to merge. This discipline is itself a contribution, because the decision log and the test-evidence log it produced give the claims in this dissertation an audit trail that can be checked against the repository.

5.10 Representative listing

The scorer illustrates the trace-driven, pure-core principle from the design. It takes a scenario, a trace and the source, and returns a structured result with no input, no output and no hidden state, which is exactly what makes it testable with synthetic traces.

```
@dataclass(frozen=True, slots=True)
class GradeResult:
    passed: bool
    reasons: list[str]    # one user-facing line per failed criterion
    score: int            # 0 to 100

def grade(lesson: Lesson, tracer: Tracer, source: str = "") -> GradeResult:
    aggregates = _compute_aggregates(tracer.events())
    reasons: list[str] = []
    for criterion in lesson.success_criteria:
        reason = _check_criterion(criterion, aggregates, lesson, source)
        if reason is not None:
            reasons.append(reason)
    score = max(0, 100 - SCORE_PENALTY_PER_FAILURE * len(reasons))
    return GradeResult(passed=not reasons, reasons=reasons, score=score)
```

5.11 Showcases, icons, and project files

Three showcase programs ship as example tabs and double as living documentation of the command surface, each verified to run in the interpreter QA sweep. Encore is the flagship: functions, loops, nested loops, counters and pen geometry over the base command set alone, so it runs on every build and shows the language working. Searchlight is a battery-managed

survey and rescue mission, a robot given real work, running a systems check and patrolling a search area with every drive guarded by the distance sensor so it never touches a wall. Gauntlet is a capability reference that draws its own proof, exercising recursion, nested loops, chained comparisons, list building and a run of pen geometry. Around these, the interface chrome was professionalised: a procedural SVG icon set replaces every emoji-as-icon, each icon a handful of primitives built offline against `currentColor` so it renders identically on every operating system, which the interpreter QA enforces by asserting zero emoji glyphs remain in the chrome source. A whole project, the build, the programs, the active world, the render settings and the accumulated memory, serialises to and from a single `.kodro` JSON file with bounded sizes and a validator that never throws, so a session is portable and a backup is a file copy.

5.12 Removing the voice route

An earlier build offered a third way to program, a voice route with speech-to-text and a spoken assistant. It was removed in full: the voice agent, the microphone input, the text-to-speech, the voice settings and the Python voice modules are gone, and a search of the shipped web source finds no speech recognition, no microphone access and no voice-to-code path. The visual `say` command remains, but it is a console line the robot writes, not a spoken output and not a voice input. The decision was a scope one made honestly: the voice path could not be made reliable and cross-platform under the offline constraint without an optional component the design did not want to depend on, so rather than ship a fragile feature it was cut, and this dissertation describes the two ways to program that actually ship.

5.13 Two front-ends over one engine

The interface exists in two forms over the same engine. The original is a Tkinter interface that ships with Python and renders on any machine, retained as the guaranteed-offline fallback. After formative review found the Tkinter look dated for an audience that benchmarks software against web and mobile apps, a second front-end was added in the medium the reference design was authored in, a vendored offline React interface rendered in a pywebview window. Both front-ends drive the same engine, scorer, library and store. The honest risk here, recorded in the next chapter, is the dual interpreter: the web interface ships its own JavaScript interpreter for instant animation, whose surface differs from the Python API, and the planned convergence is to stream real engine events to the view and retire the second interpreter.

5.14 Challenges met and fixed

These are real defects, found and fixed, and they make the strongest reflection points.

- **A procedural surface over a stateful engine.** A flat set of free functions has to reach a single live engine. This was solved with a module-level binding that the functions consult at call time, which also let the headless tests and the scorer drive the engine with no interface present.
- **An open loop.** An early build ran the user's program but never responded to it, with no pass or fail and no feedback. The fix wired scoring, hinting, persistence and the verdict into a single finish step at the end of every run, which a structured review had identified as the single biggest weakness of that build.
- **A silently detached trace.** While fixing the loop, the active trace could be detached between runs, so a run recorded zero events and some integration tests passed vacuously. The fix re-asserts the trace and engine bindings at the start of every world reset, and the tests

were strengthened to prove real movement, since the battery only drains if the robot actually moved.

- **Four kinematic replicas that had drifted.** The studio, the Python engine, the scenario validator and the self-test each carried their own copy of the motion arithmetic, and they had drifted until they disagreed by an order of magnitude on battery cost per metre. The fix collapsed them onto the one shared motion model and locked it with the constants-hash, golden-trace and formula-parity gates, so the number the fidelity badges rest on cannot differ between engines again.
- **Two honesty defects in the fidelity surfaces.** A panel review of the version 2.0 build found the two most damaging kind of defect for a proving ground, a sim that tells the user a false thing about a real build. A slow imported robot was simulated faster than its real top speed while still badged honoured, with warning copy that named the wrong direction; and the predictive design check used a different mobility model from the live simulation, so the two headline surfaces contradicted each other on the shipped reference robot. Both were fixed: the top-speed badge now drops to approximated and states the true direction when the speed falls outside the simulable band, and the design check drives its mobility verdict from the same force-ratio model the simulation uses when an imported build is present. These are recorded here rather than buried because they are the clearest evidence that the honesty discipline is enforced by review and repair, not merely asserted.
- **Headless interface fragility.** A modal error dialog blocked forever on the headless test runners, once burning a job to the six-hour ceiling. A per-test timeout was added so any hang fails fast with a traceback, which then pinpointed further headless issues and led to treating one platform's interface tests as informational while the other two remain hard gates.

5.15 Packaging

The desktop binary is built with PyInstaller, which bundles the interpreter, the runtime, the libraries and the asset bundle into a single windowed application, so deployment is one download with no separate runtime install. The build is reproducible from a specification file and was smoke-tested on a real desktop, where the packaged application starts cleanly and renders the full interface, which the headless suite otherwise exercises without a display.

5.16 Web deployment

The desktop application remains the full-featured path, but the same vendored interface now also ships as a zero-install web build. A flag on the bundle build script emits a fully self-contained static site, plain files with no server-side component, which a continuous-integration workflow deploys to GitHub Pages, so the studio can be tried from a link without installing anything. The offline character survives the move in two ways. First, the build is a progressive web application: a manifest and a service worker precache the application shell, so after the first load the site keeps working with the network off. The service worker serves same-origin requests cache-first and never touches a cross-origin request, so a call to a user's own optional cloud key or to a localhost Ollama server can never be intercepted, cached or leaked by the caching layer. Second, the privacy claim is enforced by a gate rather than asserted: a dedicated check builds the static site, serves it from the loopback address, loads it in a headless browser under a network log, and fails if the default page makes a single request to any external host, alongside a proof that the application actually boots.

What runs where is stated precisely, because the two builds are not yet equivalent. The whole simulation core is client-side JavaScript and therefore identical in the browser: the interpreter,

the randomised validation, the worlds and the three-dimensional viewport, the Robot Lab with specification import and export and the verification report through the browser's own file pickers and downloads, the self-refinement memory, and the assistant through a provider layer that stays offline-first and keeps the same deterministic self-test gate in front of generated code. The lesson library is ported: the browser build fetches a lessons file generated by the same serialiser the desktop bridge uses, and a parity test fails the suite whenever the shipped file is not byte-identical to a fresh export. What stays desktop-only at the time of writing is the graded classroom loop: grading a lesson attempt, the hint engine, pupil records and the teacher report all cross the Python bridge, and in the browser those calls degrade to a clean no-op, so the web build presents lessons to read but does not yet score them. Porting the trace grader to JavaScript, parity-checked against the same golden lesson traces the Python grader is tested on, is the defined next slice of this work.

Chapter 6

Evaluation

This chapter records how Kodro was evaluated. It separates what was actually measured, the automated verification and an analyst-run formative review, from what is planned but not yet done, a study with real users. Nothing here reports data from human subjects, because that study has deliberately been left for a point at which it can be run properly under consent.

6.1 Evaluation strategy

Three complementary methods were chosen, in increasing cost and decreasing frequency.

Method	What it checks	Status
Python test suite	Correctness of every subsystem, regression safety	Done, continuous
Interpreter QA harness	Interpreter, kinematics and product coherence end to end	Done, continuous
World and interface nets	Site identity across worlds, interface flows and modals	Done, continuous
Persona review	Whole-product usability across user types	Done, formative
Persona-task evaluation	Assistant code generation and the self-test safety net, scored by execution	Done, deterministic
Adversarial persona panel	Reproducible code defects across simulated makers, verified against source	Done, seven rounds
KodroBench grounding benchmark	Invented-symbol rate and seeded task success of local models	Done, v0.1 measured
User and refinement study	Real efficacy and the refinement loop	Planned, future work

Table 6.1. The evaluation strategy and the status of each method. The last row is not yet run.

6.2 Automated verification

The primary evidence of correctness is the Python suite, which grew with the codebase and gated every change. The Python engine carries more than one thousand tests, unit, property-based and headless integration, gated at a minimum of eighty five percent line and branch coverage that every push must clear, run on continuous integration; at the time of writing the suite measures eighty nine percent against that gate. These tests pin the behaviour that matters: the procedural API never raises and clamps bad input; the physics, the sensors and the scorer are deterministic; the sandbox rejects unsafe code and enforces a hard timeout; the shared motion model's constants, catalogue traces and physical closed forms are locked against the studio twin by the three conformance gates; and the end-to-end run, score, hint, persist and reward loop is exercised on the fully wired application, so the loop is proven to close rather than asserted to.

Alongside the Python suite, a separate interpreter QA harness drives the shipped interpreter with the real kinematics and the wall ray, and asserts the command semantics, every bundled example program including the three showcases, and a set of product-coherence checks. It passes at one hundred and eighty of one hundred and eighty. Every example terminates, moves, stays inside the arena, never hits a wall and never throws, and the command semantics, the two motion call styles, the turn, the speed clamp, the guarded huge exponent, the loops and the sensor reads, are each asserted, as are the interpreter diagnostics, the honesty of the design-check command list and the absence of emoji glyphs from the interface chrome. This harness is the evidence that the user-facing program surface behaves, independently of the Python engine.

Two headless browser nets cover the interactive surface, which the deterministic suites by design do not reach. A world net loads every base world and all seventeen named mission sites and asserts each stamps its own site treatment in the rendered scene, so a site that fails to build as itself fails the net. An interface net exercises the studio: six end-to-end flows, a set of behaviour assertions covering command gating, run feedback, the studio and classroom modes, the specification import, the fidelity card and the showcase run, and the opening of every toolbar modal. Both nets render the real WebGL scene in a headless browser through a software rasteriser, so the proof needs no graphics card and no network. These nets are deliberately reported as a smoke signal rather than a hard gate, because they depend on a headless browser spawning reliably: on a loaded machine a browser launch can time out, which fails an assertion for reasons that have nothing to do with the product, so the nets never break the build on a spawn hiccup and their value is in catching a real regression when the browser does run. In a clean run the interface flows and the world sites pass; the value of the method is precisely that it targets the integration seams the deterministic suites leave uncovered.

These nets earned their place by catching defects the unit tests could not, because those defects live in the wiring rather than in a pure function: selecting a real-world site resolved through the wrong table and crashed the view; the three-dimensional robot mesh faced ninety degrees off its travel direction while the physics stayed correct; the parts catalogue advertised commands the interpreter does not implement; and several controls fell below the accessible-contrast and touch-target thresholds. Each confirmed finding was fixed and then re-verified against the net.

6.3 Persona review (formative)

To evaluate the whole product rather than its units, a structured review was conducted across simulated personas spanning the range of makers who might use the tool. Each persona drove a session and returned a mark out of ten with the concrete defects it found; after each round the named defects were fixed and the build was re-rated. Table 6.2 reports the measured trajectory.

Round	Personas	Focus	Mean (/10)
1	50	First contact with the build	5.86
2	12	After the first round of fixes	6.50
3	50	After wiring the model and tools into the app	7.10
4	50	After grounded answers were added	7.36
5	8	Focused review of the new three-dimensional world	6.40

Table 6.2. Persona review trajectory across rounds. The figures are reported as measured by the analyst-run method, with the threats to validity stated below.

6.4 Objective persona-task evaluation

The persona review above is a subjective inspection, and its single-analyst scoring is its weakest point. To put a deterministic, reproducible number on the question that most affects a non-expert, whether the offline assistant turns a plain-language request into a program that actually works, a second persona evaluation was run that removes the analyst from the scoring entirely. Four simulated personas, a beginner with no code, a teacher preparing a class demonstration, a precise hobbyist maker, and an accessibility-focused low-vision user, each attempt five tasks, phrased in that persona's own words: drive forward, turn and move, drive a square, stop before an obstacle using the distance sensor, and a counted loop. The local assistant model answers; the result is judged not by any model and not by any human but by the same vendored interpreter and self-test that the product ships, over up to three correction turns in which the self-test's own summary is fed back the way a user would report it. Success is execution, not opinion. The harness is the script `scripts/qa_personas.mjs` and is fully offline, with the local model as its only peer. Table 6.3 reports the measured funnel over the twenty persona-task cells.

Outcome (over 20 persona × task cells)	Cells (%)
Compiled (valid program produced)	17/20 (85%)
Ran clean through the interpreter	15/20 (75%)
Stayed inside the arena (no wall hit)	7/20 (35%)
Completed the task	7/20 (35%)

Table 6.3. Objective persona-task funnel, model `kodro-coder`, up to three correction turns. Each outcome is judged by the shipped interpreter and self-test, not by a rater. Mean turns to success was 1.0 (every success landed on the first attempt; no failing cell recovered across the correction turns). The figures are from a single evaluation run of a stochastic model.

Persona	Task complete	Task	Complete
Beginner (no code)	0/5 (0%)	Forward	2/4 (50%)
Teacher (class demo)	1/5 (20%)	Turn and move	1/4 (25%)
Maker (precise)	3/5 (60%)	Square	0/4 (0%)
Low-vision (accessibility)	3/5 (60%)	Obstacle stop	1/4 (25%)
		Counted loop	3/4 (75%)

Table 6.4. Task completion broken down by persona and by task.

The reading is candid and it cuts both ways. The assistant produces syntactically valid code, eighty five percent compiled, that mostly runs, seventy five percent, which is the floor a beginner needs in order not to be stranded on a syntax error. Task-correct behaviour is limited but real: thirty five percent of cells completed the task in the single reported evaluation run; the precise maker and the accessibility-focused phrasing each succeeded at sixty percent against zero to twenty percent for the beginner and teacher voices; and the counted loop succeeded for three of the four personas. This is the honest ceiling of a small model in the one-to-four-billion-parameter range running on a laptop with no cloud, and it is reported as such rather than hidden.

The figures above are themselves the product of iterations the evaluation drove, which is the clearest evidence that the method is useful rather than decorative. The first run scored only thirty percent task completion and, more tellingly, just ten percent of programs stayed inside the arena: the model systematically over-shot distances, reading "a few metres" as a thirty-metre move. Feeding that finding back as a single concrete change to the assistant's grounding, stating

that the arena is small and a normal move is one to five metres, and giving the loop and sensor patterns for the harder tasks, lifted the in-arena rate from ten percent into the thirty five to forty percent band and took the sensor-gated stop from never completing to completing in a quarter of cells. A second iteration fixed the measurement itself: a stronger code extractor stopped stray markdown backticks in truncated replies from failing compilation unfairly, lifting the compiled rate to eighty five percent without touching the model. Because a sampling model is stochastic even at low temperature, repeated runs of the same harness vary, so the headline completion figure is honestly a single-run number rather than a mean over repeats; the table reports the run whose full results are committed alongside the code, not a best one.

The result that matters most for the design is the third row read against the second. Even after the grounding fix, of the fifteen programs that ran, eight would still have driven the robot into the arena wall. The deterministic self-test caught every one of these before it reached the learner and returned an actionable correction rather than a crash. The evaluation therefore validates the central architectural choice directly: the small offline model is not fully trustworthy on its own even when well grounded, which is exactly why Kodro pairs it with a deterministic execution check instead of surfacing raw generated code. The weakness of the model and the value of the safety net are one finding, not two.

6.5 Adversarial code-verified persona panel

The two persona methods above ask about opinion and about task execution. A third method asks a different and more falsifiable question: can a large adversarial panel of simulated makers find real, reproducible defects in the shipped code, and does fixing what it finds converge? Unlike the formative review, nothing here is a mark out of ten, and unlike the objective persona-task evaluation, nothing here is scored by whether a generated program runs. Each persona is instead pointed at the source with a single instruction, to try to break the tool and to distrust every number it prints, and every issue it reports is then put through an adversarial verification pass that defaults to not-real and only accepts a defect that reproduces end to end against the shipped files. The whole panel is run offline as a fan-out of local analysis agents, and its output is a list of confirmed, code-cited defects rather than a score. Each confirmed defect was accepted only with a file-and-line citation that reproduced against the shipped files, and the fixes that closed the confirmed defects are recorded in the project's commit history.

The panel was run in seven rounds, each on the build the previous round's fixes produced, and the arc of the rounds matters more than any single finding. The first round put thirty personas against the v2.0 build and, after adversarial verification and de-duplication, produced four confirmed defects, all in the fidelity-badge disclosure; the strongest was a genuine honesty bug in which a top speed clamped into the simulable band was still labelled "honoured" by the live readout and the exported report, even though the schema had computed the correct, lower-confidence tier all along. Because the obvious risk with any find-and-fix pass is that it converges on the one area it happened to look at, the second round aimed twenty eight fresh personas deliberately away from the badges, at keyboard, screen-reader, contrast, offline-guarantee, import-fuzzing, determinism, mobile and reduced-motion surfaces, and it did not come back empty: it produced thirteen further confirmed defects, five of them high severity, led by a persistent boot failure in which a corrupted project file threw at start-up on every reload until browser storage was cleared by hand, alongside a keyboard focus trap and motion that ignored the reduced-motion setting. A third round widened the aperture to the Python engine and its sandbox, the two engines cross-checked formula by formula, the assistant's grounding path, the memory store and packaging; sixteen of its thirty lenses came back empty, and the rest produced sixteen further confirmed defects, two of them correctness bugs in the grading layer itself: a grader that counted

the distance a program commanded rather than the distance the rover actually travelled, so a rover clamped against a wall could pass a distance requirement it never met, and environment sensors hardcoded to Earth values, so a program that branched on gravity or temperature was graded on the wrong world. Every confirmed defect in these three rounds was fixed and the automated gates were re-run to green.

The fourth and fifth rounds swept the remaining corners with thirty personas each, from the replay dialog and the hint engine to every Tk panel and all eighteen lessons. Half of the fourth round's lenses came back clean, and its twenty-three verified findings were, with one exception, maintainability and polish rather than correctness, the exception being in-progress lesson code that lived only in memory and was lost on refresh; twenty were fixed, two interpreter language changes were recorded as deliberate limitations of the restricted, golden-trace-gated teaching subset rather than chased, and one reported defect proved on implementation to be intended behaviour. The fifth round returned the most clean lenses yet, twenty of thirty, and most of its fifteen findings were polish, but it corrected any impression that the tail holds only trivia. One finding had been latent since the first commit: the grader was never actually enforcing the no-collisions criterion, because it counted collisions from an event stream the engine does not emit while the real count accumulates on the rover snapshot, so a program that drove into a wall passed a no-collisions lesson with full marks, and four rounds had walked past it because the only tests touching it hand-built synthetic events the engine never produces. Six further findings were deferred with stated reasons and one reported item was already correct.

The sixth round changed tactics: rather than sweep blind, it pointed a third of its thirty personas at a single known bug class, since the round-three commanded-distance bug and the round-five collision bug shared one shape, an aggregate read from the wrong source instead of the real number the engine records. Tracing every metric the grader, hint engine, report and achievements compute back to its source found a third and a fourth copy of the same wrong-source pattern that five rounds had missed, one in the fallback application's own collision count and one in the hint engine's distance, alongside two camera animations still ungated by the reduced-motion contract and an interpreter parity gap; all eight were fixed. The seventh round audited a new feature on the day it shipped, the optional bring-your-own-key path that lets a user connect their own cloud model alongside the offline default, with sixteen personas weighted toward the security of the new surface. The security verdict was clean: no request can reach a non-local host with the default provider or with no key set, the key is never logged and is sent only to the provider the user picks, and cloud-generated code passes the same deterministic self-test gate as local code before it can reach the learner. Thirteen of the sixteen lenses came back clean, and the six confirmed findings were all honesty of labelling rather than security, panels and documentation that still claimed fully local answers while a cloud key was answering; all six were fixed the same day.

That each pass on a freshly fixed build kept surfacing real defects is the clearest evidence that this method is not a rubber stamp, and it is also a plain reminder that a passing test suite is not the same thing as a sound product. Across the seven rounds, one hundred and ninety four simulated personas found eighty five confirmed defects, of which seventy five were fixed, eight were deferred with stated reasons, and two were reclassified or found already correct; the share of adversarial lenses that came back clean rose from none in the first round to over three-quarters by the sixth and above eighty percent in the seventh, and the sixth round's clearest lesson was methodological, that hunting a known bug class directly finds instances a broad sweep walks past, the wrong-source aggregate alone turning out to have four copies scattered across the codebase. Both facts hold at once, and together they are the closest thing to a convergence signal this method can give: the code became demonstrably more trustworthy under adversarial reading, and the testing never reached a point where the product could be declared done. The honest

limits are the same ones that apply to every simulated method in this chapter, and they are worth stating rather than glossing: these are language-model personas and not people, the panel measures the code and not a learner, and a clean round is evidence of nothing more than that a particular set of adversarial lenses found nothing that reproduced. It is a cheap and repeatable net that keeps catching real bugs before a human has to, which is precisely why it earns a place next to the deterministic suites and precisely why it cannot be read as validation. It does not license a perfect score, and the human study remains the right next step.

6.6 Measuring grounded code generation (KodroBench)

The methods above evaluate the product. A final method measures, in isolation and with a number, the specific failure the grounding design exists to prevent: a language model emitting a program symbol that the built robot does not expose. The distinction from prior art is worth stating carefully. The nearest neighbour is RoboEval with its CodeBotler harness (Hu et al., 2023), which scores language-model programs for service robots by executing them and checking temporal properties over the resulting traces. Its programs, however, are written against one fixed robot interface, so the characteristic hallucination it can surface is an invalid argument passed to a valid primitive, for example a navigation call to a location that does not exist. General benchmarks of interface hallucination in ordinary software measure invented calls but involve no robot and no execution of the offending program. What none of the surveyed work measures is an invented *symbol* against a *per-build* interface. Because Kodro derives the command set from the user's build, the ground truth for what counts as a valid call is per-design rather than fixed: a model that is perfectly grounded for one robot can invent for the next, and the same call is valid on one build and hallucinated on another. KodroBench was built to measure exactly that intersection.

The metric is deliberately small and deterministic. The grounding checker parses a generated program into its abstract syntax tree and classifies each call it finds. A bare-name call is invented when it names neither a command in the build's fitted-command set nor one of the three builtins the sandbox exposes, unless the program itself defines that name or aliases it from a fitted command. An attribute call is invented when it is made on an object the program never created, since the fitted API is a set of bare functions and exposes no objects: a call such as `rover.forward()` is therefore counted, and only a real built-in container or string method on a name bound to a literal container, for example `xs.append(1)` after `xs = []`, is exempt. That attribute rule is what lets the metric catch an entire object-style command surface rather than waving it through as ordinary Python. A syntax error is recorded as a result rather than raised as a failure, and the function is pure, so the same program and fitted set always return the same verdict. Around it, the benchmark harness gives each model a natural-language instruction together with exactly the commands the build exposes, then runs the generated program across ten seeded, domain-randomised runs through the same Python engine the desktop application uses, following the randomisation argument of Tobin et al. (2017); a run succeeds when the program moves at least the task's minimum distance without a collision, and the reported success is the mean over seeds and tasks. The harness also supports the unbiased pass@k estimator of Chen et al. (2021) when several independent samples are drawn per task, and carries a development and held-out task split whose two held-out tasks are adversarial, in that the instruction names a capability the build lacks, for example a build with no distance sensor. With five tasks this split is a mechanism rather than a rigorous held-out claim, and it is reported as such; growing the suite toward twenty to fifty tasks is what would make it meaningful. The results are written as a machine-readable record and the leaderboard is generated from that record, never typed by hand, so every number in Table 6.5 traces to a run. A deterministic floor, a fixed grounded program

per task, is byte-reproducible and gives continuous integration a value to assert against.

Model	success@N	Invention rate	Syntax errors	Collisions
Deterministic floor	0.22	0.00	0.00	1.24
`gemma3:4b`	0.24	0.00	0.20	0.56
`gemma3:1b`	0.02	0.00	0.20	0.18
`llama3.2:3b`	0.00	0.00	0.00	1.44
`kodro-fast` (local fine-tune)	0.00	0.60	0.40	0.00
`kodro-coder` (local fine-tune)	0.00	1.00	0.00	0.00

Table 6.5. KodroBench v0.1 (five tasks, ten seeds per task). success@N is mean seeded task success; invention rate is the fraction of tasks whose program calls a symbol outside the build's fitted-command set; syntax errors and collisions are the per-task rates. Values are as recorded in the versioned results file.

The headline finding is honest and it is not the one that was hoped for. The two models that invent are the project's own local fine-tunes, produced earlier in the project specifically for the assistant role. `kodro-coder` emits an object-style `rover.forward()` call surface on every one of the five tasks; against each task's fitted-command set, which for the benchmark's Python engine is a set of bare functions with no `rover` object, every such call falls outside the fitted set and the grounding checker flags it, giving an invention rate of 1.00 and a task success of 0.00. One honest qualification belongs here: the web studio's own interpreter additionally accepts the object-style `rover.forward()` form as a documented compatibility alias for the bare verbs, so that surface is not universally rejected across the product, and what the benchmark measures is invention against the per-build fitted set it prompts the model with, on which the object surface is ungrounded. `kodro-fast` invents on three tasks of five and drifts into unrelated prose on the other two. The general small models never invent a symbol: given the fitted-command list in the prompt, they stay inside it, and their failures are weaker ones, malformed indentation, missing arguments, or a plausible name used as a bare statement, so their programs mostly parse but rarely complete the task. The zero collision rate of the two fine-tunes is failure rather than caution, since a program built from invented calls never drives the robot at all. And the deterministic floor completing only 0.22 says the seeded obstacle field is genuinely hard for a fixed open-loop program, which keeps the model rows in proportion.

Two readings follow, and both matter to the design. First, the metric works: it caught, at scale and mechanically, a failure mode of this project's own artefacts that a compile check or an opinion score would have missed, an entire invented command surface, and the result is reported as measured precisely because it cuts against the project's own fine-tunes. Second, the benchmark localises the failure precisely: every invented call in the table is a call to a symbol outside the task's fitted-command set, which is exactly the class of mistake the grounding metric exists to name. Whether a given surface is then refused at run time depends on which engine runs it, and the honest statement is narrower than a blanket vindication: on the benchmark's Python engine the fitted set contains no `rover` object, so the object-style calls are ungrounded there, whereas the web studio interpreter accepts the `rover.forward()` alias and would run the same program. The benchmark therefore quantifies the ungrounded-symbol failure the metric was built to measure, without claiming the product refuses every such call on every surface.

The threats to validity are stated plainly. Five tasks are a proof of mechanism, not a benchmark of record. The success predicate is a coarse motion criterion, moving far enough without collision, rather than a task-semantic judgement. The runs are simulation only, on a single machine, over a handful of small local models, and the table reports a single-sample run rather than a pass@k average. The two fine-tunes were trained by the same author, so their failure is evidence about

these specific artefacts, not about fine-tuning in general. What survives those limits is still useful: a deterministic, seeded measurement that needs no graphics card, runs in continuous integration, and puts a number on an axis the surveyed prior art leaves unoccupied.

6.7 Threats to validity

The persona review is a low-cost inspection method in the tradition of heuristic evaluation, and it is reported with its limits stated plainly rather than dressed up as user evidence. Three threats matter. First, a single analyst simulated every persona, so the scores carry that analyst's bias and the inter-rater reliability is unknown. Second, the personas are informed guesses about real makers and cannot capture genuine confusion, motivation or the messiness of real use. Third, re-scoring a build the analyst made risks optimism, so a rising mean should be read as a direction of travel that the tool became more learnable to drive, which is a usability signal and nothing more. The fifth round, scoring the new three-dimensional world lower than the fourth, is left in deliberately, because the honest record includes the rounds that went down as well as the rounds that went up. These threats are exactly what the planned user study is designed to address. The objective persona-task evaluation in the previous section removes the rating bias, because nothing it reports is scored by a person, but it carries its own limits and they are stated with equal plainness: its personas are still simulated phrasings rather than real users; its task predicates are coarse proxies for success rather than a learner's judgement; and it exercises one model in one configuration on five short tasks, so it speaks to the assistant's code generation and safety net, not to whether a person actually learns. It too points to the same human study, and neither method is offered as a substitute for it.

6.8 The planned studies

Three studies are specified and left for a point at which they can be run under consent. The first asks whether the tool helps: a within-subject design with about sixteen non-experts, each attempting two comparable held-out scenarios, one with Kodro's grounded help and reviewer and one without, with order counterbalanced, where success is a design that completes the scenario in at least sixteen of twenty randomised runs, and iterations and collisions are recorded alongside. The second asks whether the tool refines: the same participant tackles a sequence of related scenarios in two arms whose only difference is the memory, the full arm against an ablated arm with reflection retrieval and the skill library turned off, with the full arm expected to reach a working design in at least a quarter fewer iterations. The third measures the assistant directly on a fixed set of design questions including adversarial and off-specification ones, where grounded answers must be correct at least eighty five percent of the time and invent a missing part less than five percent of the time, and where grounding must beat the same model asked without the specification, with the five percent invention rate set as a release gate above which only the deterministic path ships. The analysis for each is fixed before any data is collected, and a positive result is to be read as a signal to confirm at scale rather than as proof, because the samples are small and convenient.

6.9 A speculative teacher-persona walkthrough

One further design artefact sits alongside these plans and must not be confused with them. To surface classroom deployment requirements ahead of the human study, a speculative walkthrough was written from the point of view of eight UK secondary teacher archetypes, from a non-specialist covering Computing to an experienced GCSE lead and a SENDCo, and it is kept with the teacher documentation, labelled as speculation. It is an analyst inspection in the same tradition as

the persona review and a weaker one, because nothing in it was run or measured: it involved no participants, no timings and no model-generated scores, and it produced design hypotheses rather than findings. Its value is the requirements it raised, which now sit on the roadmap: hints should be toggleable off for summative assessment, because a hint engine that cannot be disabled undermines controlled conditions; lesson material should map explicitly to exam-board terminology rather than stopping at the national programme of study; and the offline single-executable deployment is likely to matter as much for school network filtering and data-protection logistics as it does for the design goals already argued. Whether any of this holds in a real classroom is exactly what the planned study asks, and no time-on-task or satisfaction figure exists until that study runs.

6.10 Instrumentation

The application is already instrumented for these studies without any cloud service. Every run is recorded with its program, trace, score, battery and collisions, the reflections and skills accumulate in the local store, and the results can be exported on demand, all on device. This is consistent with the offline constraint and it simplifies the ethics and data-protection position for any future study, because no personal data ever leaves the machine.

6.11 Summary of the evaluation

The honest position is that the product has been verified, it does what it claims and the tests prove it, and it has been heuristically reviewed, it should work for a range of makers and builders, but it has not yet been validated with real users. The interpreter QA passes at one hundred and eighty of one hundred and eighty, the engine suite passes at scale with high coverage above the eighty-five percent the suite enforces, the world and interface nets exercise the interactive surface, and the persona review traces a real improvement as defects were fixed. An objective persona-task evaluation, scored only by the shipped interpreter and self-test, then puts a deterministic figure on the offline assistant: it produces running code and, after a grounding fix the evaluation itself prompted, completes the task in roughly a third of cells (thirty five percent in the single recorded evaluation run), and its chief result is that the deterministic self-test caught every unsafe program the model still produced before it reached the learner, which is the safety argument the whole design rests on. A third, adversarial persona panel of one hundred and ninety four simulated makers across seven rounds, each finding verified against the source before it was accepted, found eighty five confirmed defects and drove the correction of seventy five of them, including a grader that never actually enforced its no-collisions criterion, four copies of a single wrong-source aggregate bug scattered across the grader, hint engine and fallback app, a persistent boot failure from a corrupted project file, a keyboard focus trap, motion that ignored the reduced-motion setting and several honesty defects in the fidelity badges; that a sixth round, on a much-fixed build, still surfaced high-severity bugs by hunting a known bug class directly, even as over three-quarters of its lenses came back clean, is the plainest evidence that a passing suite is not a sound product, and it too is offered as a bug-finding net rather than as validation. A seeded grounding benchmark, KodroBench, adds a measured warning from the opposite direction: the project's own local fine-tunes invent a command surface the build does not expose, on up to every task, while the general small models stay grounded but weak, which quantifies at the level of the model exactly the failure the deterministic gate exists to stop. The teacher-persona walkthrough adds design hypotheses for the classroom, not evidence. The remaining step to a confident claim of usefulness is the human study, for which the instrumentation is in place, and the next chapter is candid about the limitations that study and the roadmap will have to address.

Chapter 7

Discussion and Limitations

A dissertation that only reports what works is not trustworthy. This chapter sets out, plainly, what Kodro does not do yet, so that a reader and a future contributor can plan around the gaps. Each limitation is framed as something the design is working toward rather than something broken with no path forward, and each is labelled by honest status: working with a known gap, experimental, or roadmap.

7.1 Motion is kinematic, not rigid body

The most important limitation to be clear about is the one most easily oversold. The robot and every moving agent advance through a hand-written kinematic tick. There is no rigid-body solver, no contact forces and no friction model resolving the motion in the runtime. A car banks and a rover throws its weight around because the per-type motion code fakes the feel convincingly, not because a physics engine resolves forces, and collisions are detected as overlap events and recorded rather than resolved with impulses. This is a deliberate trade that buys a believable feel at a laptop-friendly cost, and it is the right trade for a non-expert judging a behaviour. It is not the right trade for deployment-grade validation, and the dissertation does not claim it is. The roadmap item that addresses this is the wiring of a real rigid-body engine into the production runtime.

7.2 The physics engine exists but is not on the hot path

Related to the above, the Python engine ships a tested wrapper around a real two-dimensional physics library that builds boundary walls, places the robot and obstacles, and records collisions. It is exercised by the Python suite and used by the Python scorer. It is not, however, what the visible studio runs, because the shipped Studio interface steps the world through the vendored kinematic interpreter. So the rigid-body path is real code, but it is not on the hot path the user sees when they press run. Making that wrapper the source of truth for the visible motion is the single largest change on the roadmap, because it touches the interpreter, the 3D Viewport and the scorer at once.

7.3 Sensor command coverage is partial

The grounding guarantee that the design rests on is fully enforced today for the ultrasonic sensor (the distance command) and the inertial unit (the heading and tilt commands), which are the commands the interpreter actually implements; the scan command is gated on the same ultrasonic part. The remaining parts, the positioning sensor, the camera, the bumper and the gripper, are real fitted hardware that change the build but carry no implemented command yet, so they are deliberately not advertised as callable rather than offered as a sensor a build cannot use. This is the gap between the design's promise and the current code, and it is named here rather than glossed, because the whole safety argument depends on closing it. Full gating across every surface is a near-term roadmap item, and it is the first thing the realism work should finish.

7.4 Procedural geometry, with surface shading and a gated post pass

The 3D Viewport has no asset loader for any interchange format, so it cannot import a real robot description or a modelled part; everything is procedural geometry. This part of the limitation is genuine and stands. The shading side, however, is now mitigated rather than absolute. Each open-terrain ground builds a tileable normal map, derived by a Sobel pass over a periodic height field, together with a roughness map, both generated in canvas at scene-build time, so the light grazes real micro-relief instead of a flat coloured plane. The Cinematic quality tier adds a small, hand-written, fully offline post chain, a bloom built from a bright-pass and a separable Gaussian blur composited additively over the base render, with a vignette. It is written directly against the core renderer because the library ships its post-processing only in an examples tree that is not vendored and cannot be fetched under the offline rule, and it is gated to the Cinematic tier, disabled under a reduced-motion preference, turned off automatically if the slow-hardware downgrade fires, and dropped to the plain render if the device cannot allocate its targets, so the offline and low-hardware guarantees are not weakened. What remains genuinely absent is a glTF or similar asset loader and the heavier effects, ambient occlusion, depth of field and motion blur. The loader stays on the roadmap rather than shipped because neither it nor good interchange models can be obtained under the zero-network constraint in this environment, and that is recorded as future work rather than presented as a gap that was quietly skipped.

7.5 An arena-scale mismatch between the two engines

The in-browser interpreter runs the robot in a thirty-metre arena while the Python engine and scorer work in a ten-metre world. Every bundled example runs and the conformance check passes, because the criteria are scale tolerant, but a program tuned visually in the web interface can score differently under the Python scorer. The metres-to-centimetres scaling in the interpreter mitigates this for motion but not for boundary behaviour. A single shared arena definition that both engines read is the fix, and it is recorded as a known issue rather than left for a reader to discover.

7.6 The measured-build simulation is the studio only

This is the most important scope limit of the fidelity claim, and it is stated bluntly. The one shared motion model locks the two engines at three levels, the constants, the catalogue traces, and now the physical closed forms, so an imported build's derived numbers, its top speed, stall force, acceleration, energy, runtime, turn timing, stopping distance and sensor pose, are provably identical in the studio and in the Python engine. What is not yet shared is the full end-to-end simulation of an imported build. The Python engine's sensor rays still originate at the robot's centre and use fixed module ranges, so they do not consume an imported sensor's mount pose or its range, and the public Python API does not read an imported specification's mass, rpm or battery. The consequence is precise: a measured build is simulated end to end only in the JavaScript studio, while the Python engine reproduces the derived numbers and grades catalogue-mode motion. This is why the honoured sensor-pose and range figures are scoped in the fidelity table to the studio simulation, and why the badges never claim that the Python grader honours an imported sensor's geometry. Closing this, by wiring the sensor pose and range into the Python sensor model and teaching the public API to import a specification, is a defined roadmap item and the natural next step after the formula parity that already landed.

7.7 The small-model ceiling

Finally, the honest limit named throughout returns here. The local model is small by necessity, and a small model cannot match a frontier cloud model on raw capability. The design answers this not by pretending otherwise but by leaning on grounding and the simulation gate, so that the system is safe even when the model is weak, and useful because the help is anchored in the build rather than in the model's cleverness. The cost is that the assistant is a competent drafter and not an expert, and the deterministic path, not the model, is what the offline guarantee and the safety argument actually rest on.

Chapter 8

Professional Issues

The specification and design review asked for two things this document had folded into other sections: a separate statement of the ethical position, and an explicit account of how the project meets the BCS project criteria. Both are given here in their own right.

8.1 Ethics

No human participant took part in any study reported in this dissertation, and no personal data was collected, processed or stored at any point. The evaluation personas are language-model constructs, not people; the text labels them as simulated wherever they appear, and no conclusion about real users is drawn from them. Under the department's ethics classification the work sits in data category A0 with participant category 2, and no ethics approval was therefore required for the work as reported.

The product itself is designed so that this position holds for its users as well as for the project. Kodro runs entirely on one machine: there are no accounts, no telemetry, no analytics and no network calls in its default configuration, and the offline guard in the test suite fails the build if any shipped file reintroduces a remote dependency. A pupil's class register, run reports and saved skills live in local storage on the device and can be deleted by clearing it. For a tool aimed partly at school-age learners this is a deliberate safeguarding stance, not a technical accident: nothing a child does in Kodro leaves the room.

Two further matters belong here. First, the use of generative artificial intelligence in building the software and drafting this document is acknowledged in the Declaration, in line with University guidance, together with the boundary that every design decision and every reported result was directed and verified by me. Second, the planned study with real users, which the evaluation chapter is explicit about not having run, will require its own ethics application, informed consent from participants and agreement with my supervisor before it begins, and the instrumentation described in the evaluation chapter was built so that such a study can run without collecting anything beyond what the participant knowingly exports.

8.2 The BCS project criteria

The BCS expects a masters project to demonstrate the practical and analytical application of material from the degree, self-management, innovation or synthesis beyond routine application, and critical self-evaluation. Against each in turn. The project applies degree material practically and analytically: a recursive-descent interpreter for a Python subset, closed-form kinematics shared across two engine implementations and locked by conformance gates, property-based and coverage-gated testing, and an evaluated application of local language models under retrieval grounding. It addresses a real need in a real context: a zero-cost, fully offline design and simulation studio usable in classrooms without connectivity, budget or accounts, with the safeguarding posture stated above. The synthesis is the contribution the conclusion claims: specification import, tiered fidelity disclosure, a sandboxed simulation gate and offline self-refinement combined into one working, verified artefact rather than any single new algorithm. Self-management is evidenced by the record itself: a solo build over the project period, version controlled throughout,

gated by continuous integration on three operating systems, and steered by the supervisor's specification-and-design feedback, whose central advice, to develop the working system first and enhance it second, is the order the work actually followed. Critical self-evaluation runs through the evaluation and discussion chapters, which report the defects the reviews found, state what the simulated evaluations can and cannot claim, and decline to report validation that has not happened.

Chapter 9

Conclusion and Future Work

9.1 Summary

This project set out to build a single, offline, free desktop proving ground on which a builder imports a real robot specification, or a non-expert designs one, programs it two ways against one meaning, and validates its behaviour in a realistic, agent-populated simulation before building any hardware, with every reported figure carried at an honestly stated level of fidelity and a local model that helps without ever having the last word on safety. Against the core objectives, the artefact delivers a specification, imported or designed, that drives the simulation so that changing the build changes the behaviour, a three-tier fidelity disclosure surfaced as per-figure badges and a verification report, one shared motion model locked across the two engines by a constants hash, a golden-trace corpus and a formula-parity test, a sandboxed execution and scoring path, a validation layer of seventeen named mission sites with moving agents and randomised runs, a grounded assistant with a guaranteed deterministic fallback, and a self-refinement memory of reflections and skills, all on one machine with the network off. The interpreter QA passes at one hundred and eighty of one hundred and eighty, the engine suite passes at scale above the eighty-five percent coverage the suite enforces, the world and interface nets exercise the interactive surface, an objective execution-scored persona-task evaluation puts a reproducible figure on the offline assistant, a seeded grounding benchmark measures the invented-symbol failure directly and catches it in the project's own fine-tunes, and the persona review, a single-rater formative inspection, traces an improvement as defects were fixed.

9.2 Reflection

Four honest reflections stand out. What worked was building scoring, hinting and replay on one trace abstraction and keeping the decision cores pure, which paid off repeatedly in testability and in the ease of adding features late. What was harder than expected was headless interface testing, where a browser launch can time out on a loaded machine, so the honest outcome was to treat the browser-based world and interface nets as a smoke signal rather than a hard gate and to lean on the deterministic interpreter QA and Python suite for the load-bearing evidence. The reflection that carries the most weight for a proving ground is that honesty is not a property you assert once but one you have to enforce and repair: a panel review of the version 2.0 build found two defects in the exact fidelity surfaces the product stakes its credibility on, a slow robot shown faster than real under an honoured badge, and a predictive check that contradicted the live simulation, and only fixing both, and collapsing the four drifted kinematic replicas onto one gated shared model, made the fidelity claim true rather than aspirational. What I would do differently is to wire the feedback loop, and the single shared motion model, first rather than last, because both were the foundation the rest of the honesty rests on, and discovering their absence through review rather than by design cost rework.

9.3 Future work

The roadmap follows directly from the limitations, ordered roughly by the order the work should be taken on.

1. **Wire the imported specification through the Python engine**, teaching its sensor rays to honour an imported sensor's mount pose and range and the public API to read an imported mass, rpm and battery, so a measured build is simulated end to end on both engines rather than only in the studio. This closes the largest remaining fidelity-scope gap and is the natural next step after the formula parity that already landed.
2. **Implement and gate the remaining part commands** (the positioning sensor, the camera, the bumper, the line follower and the gripper), exactly as the ultrasonic and inertial unit commands already are, so every fitted part offers a working, gated command, closing the gap between the parts catalogue and the runnable surface.
3. **Rigid-body physics on the hot path**, making the tested physics wrapper the source of truth for the visible motion, so the 3D Viewport reflects real contact forces, friction and collision response.
4. **A single shared arena definition** so the two engines agree on the world size and a program scores the same way it looked.
5. **Richer domain randomisation**, varying obstacle placement, lighting, friction and sensor noise across runs so that a passing program is one that generalises rather than one that memorised a layout.
6. **A model-selection interface** that lists the models the local runtime reports as installed and lets the user choose the drafter and the reviewer independently, making the offline assistant legible rather than implicit.
7. **An asset and interchange import workflow** on top of the specification import that already ships: a documented path for authoring parts and terrain, and a loader for standard robot descriptions such as URDF and for glTF geometry, so published robot models can be used directly rather than only their numbers.
8. **A research bridge** to the established heavyweight simulators, exporting a Kodro build and program for high-fidelity validation and pulling the results back, which keeps Kodro light while opening a path to rigorous physics when a user needs it.
9. **A bridge to a robotics middleware** as an optional, separate component that talks to a running daemon over a local socket, so a validated Kodro program can drive real topics, recorded honestly as a long way out because it fights the offline guarantee.
10. **The human studies** specified in the evaluation, the usability study, the help study and the refinement-ablation study, run under consent with the instrumentation that is already in place, testing among other things the classroom hypotheses raised by the speculative teacher-persona walkthrough.

9.4 Closing statement

The originality of this work is not a new algorithm. Retrieval grounding, a sandboxed gate, an experience memory, a skill library and domain randomisation are each known, and the offline constraint actively limits how clever any one of them can be. The contribution is the demonstration that a real specification import, a disciplined three-tier fidelity disclosure and one shared motion model combine with these known parts into an honest proving ground that runs entirely on one machine, one where a skeptical builder sees how a real robot would perform with clarity and error bars rather than a certified verdict, and a measured, candid account of how far offline self-refinement can go when changing the model weights is off the table. The constraint is the point of the project, not a limitation to apologise for, and the remaining step to a confident claim of usefulness is the human study for which the tool is already built and instrumented.

Bibliography

Ahn, M., Brohan, A., Brown, N. et al. (2022) Do as I can, not as I say: grounding language in robotic affordances. *arXiv:2204.01691*.

Cemri, M., Pan, M.Z., Yang, S. et al. (2025) Why do multi-agent LLM systems fail? *arXiv:2503.13657*.

Chen, M., Tworek, J., Jun, H. et al. (2021) Evaluating large language models trained on code. *arXiv:2107.03374*.

dos Santos, V.G., Khadraoui, I., Farhat, I. et al. (2026) ALRM: agentic LLM for robotic manipulation. *arXiv:2601.19510*.

Hu, Z., Lucchetti, F., Schlesinger, C. et al. (2023) Deploying and evaluating LLMs to program service mobile robots. *arXiv:2311.11183*. Published as open-source software at `https://amrl.cs.utexas.edu/codebotler/`.

Huang, L., Yu, W., Ma, W. et al. (2023) A survey on hallucination in large language models. *ACM Transactions on Information Systems* (arXiv:2311.05232).

Khati, D., Rodriguez-Cardenas, D., Pantzer, P. and Poshyvanyk, D. (2026) Detecting and correcting hallucinations in LLM-generated code via deterministic AST analysis. *arXiv:2601.19106*.

Lee, Y., Song, J.Y., Kim, D. et al. (2025) Hallucination by code generation LLMs: taxonomy, benchmarks, mitigation, and challenges. *arXiv:2504.20799*.

Lewis, P., Perez, E., Piktus, A. et al. (2020) Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, pp. 9459 to 9474.

Liang, J., Huang, W., Xia, F. et al. (2023) Code as policies: language model programs for embodied control. *IEEE International Conference on Robotics and Automation*.

Papert, S. (1980) *Mindstorms: children, computers, and powerful ideas*. New York: Basic Books.

Reza, Z., Mazur, A., Dugdale, M.T. and Ray-Chaudhuri, R. (2025) Small models, big support: a local LLM framework for educator-centric content creation and assessment with RAG and CAG. *arXiv:2506.05925*.

Shinn, N., Cassano, F., Berman, E. et al. (2023) Reflexion: language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36 (arXiv:2303.11366).

Sun, T., Xu, J., Li, Y. et al. (2025) BitsAI-CR: automated code review via LLM in practice. *arXiv:2501.15134*.

Tao, Z., Lin, T.E., Chen, X. et al. (2024) A survey on self-evolution of large language models. *arXiv:2404.14387*.

Tobin, J., Fong, R., Ray, A. et al. (2017) Domain randomization for transferring deep neural networks from simulation to the real world. *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 23 to 30.

Wang, G., Xie, Y., Jiang, Y. et al. (2023) Voyager: an open-ended embodied agent with large language models. *arXiv:2305.16291*.

Wang, W., Li, Y., Jiao, L. and Yuan, J. (2026) Ro-SLM: onboard small language models for robot task planning and operation code generation. *arXiv:2604.10929*.

Wu, R., Wang, X., Mei, J. et al. (2025) EvolveR: self-evolving LLM agents through an experience-

driven lifecycle. *arXiv:2510.16079*.

Chapter A

Glossary and Abbreviations

Definitions are scoped to how each term is used in this dissertation.

Term	Meaning
API	Application programming interface; here the small procedural command surface the user's program calls.
AST	Abstract syntax tree; used by the sandbox to reject unsafe code and by the scorer to check required constructs.
Domain randomisation	Varying friction, mass, sensor noise and the scene across runs so behaviour that survives the spread is likely to survive reality.
Fidelity tier	One of honoured, approximated or not simulated; the disclosed level at which the simulation treats a reported figure.
Grounding	Constraining the model's suggestions to the build the user assembled or imported, so it cannot call a part that is not fitted.
Kinematic motion	Motion advanced by a hand-written tick that fakes the feel of weight and traction, as distinct from a rigid-body solver that resolves forces.
KRS	Kodro Robot Specification; a schema-validated JSON document of a real robot's measured numbers (mass, drive geometry, battery, sensors) that can be imported or exported and that drives the simulation through the physical closed forms.
Ollama	A local model runtime; the optional, loopback-only backend for the assistant.
RobotSpec	The structured robot specification, imported as a KRS document or derived from chosen parts; the single source of truth for behaviour and the command registry.
Sandbox	The pre-execution check that rejects unsafe imports, builtins and operations before a program runs.
Shared motion model	The single set of closed-form equations and constants, mirrored in `motion-model.js` and `motion_model.py` and locked by conformance tests, that both engines obey.
SQLite	The embedded, file-based database used for the single local store.
Trace	The recorded sequence of command events from one run; the single representation that scoring, hinting and replay all read.

Table A.1. Glossary of terms as used in this dissertation.

Chapter B

The Command Surface and a First Program

The procedural command surface the user programs against is a small set of plainly named functions, gated by the build. The motion and query commands below are representative; a command is present only when the part it depends on is fitted.

```
move_forward(distance)       # drive forward; metres in the bare style
move_backward(distance)      # drive backward
turn_left(degrees)           # turn in place to the left
turn_right(degrees)          # turn in place to the right
set_speed(percent)           # cap speed; clamped by motor and mass
stop()                       # halt
read_distance()              # only if an ultrasonic or ranging sensor is fitted
read_heading()              # only if an inertial unit is fitted
collect_sample()             # only if a collector actuator is fitted
log(message)                 # write a line to the console
```

The program that ships loaded in the editor is the first example, and it teaches the habit the whole tool is built around: look before you move. It drives forward through the default city, and whenever the range sensor reports the way ahead is not clear it steers to a clearer path instead of driving into a parked car, then prints a short report of what happened. It calls only commands the default rover carries.

```
# Drive forward, but look first. distance() reads how far the way
# ahead is clear, in centimetres, so the rover steers around a
# parked car instead of driving into it.
set_speed(60)
for step in range(14):
    if distance() < 130:
        turn_right(55)
    else:
        move_forward(1)
```

Pressing run animates the rover along the path with weight transfer and a draining battery; pressing step advances one event at a time. Changing the build, a heavier chassis or a weaker motor, changes how the same program behaves, which is the point the whole platform is built to make.